INNERMOST DEPENDENCY-CHAIN CRITERIA FOR AFP TERMINATION AND UNIVERSAL COMPUTABILITY

CARSTEN FUHS, LIYE GUO, CYNTHIA KOP

Editorial note. Source https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2025.20.
© Carsten Fuhs, Liye Guo and Cynthia Kop, CC BY4.0. Official HTML transcription with full author proof appendix and original diagrams retained.

Allowed standard prerequisites (editorial).

- Reducibility-candidate computability for accessible function-passing higher-order rewriting. Author §2 and Theorem6 (reference13) give the RC-set construction; standard computability closure facts are named in AppendixA2. All new innermost-chain and processor proof obligations are retained.

2 Preliminaries

This section collects the preliminaries from the literature. First, we recall the definition of an LCSTRS [22].111In contrast to [22] and following [21], we assume that ℓ is a pattern in every rewrite rule $\ell \to r\ [\varphi]$. For notational convenience, we also assume that $\mathrm{Var}(r) \setminus \mathrm{Var}(\ell) \subseteq \mathrm{Var}(\varphi)$, which can be guaranteed by a simple transformation and is also adopted in the second half of [22]. Then, we recollect from [13] the definition of computability (with accessibility), particularly under the strategies. We borrow several definitions from the phrasing in [21].

2.1 Logically Constrained STRSs

Terms Modulo Theories.

Given a non-empty set $\mathcal{S}$ of sorts (or base types), the set $\mathcal{T}$ of (simple) types over $\mathcal{S}$ is generated by the grammar $\mathcal{T} ::= \mathcal{S} \mid (\mathcal{T} \to \mathcal{T})$. Right-associativity is assigned to $\to$ so we can omit some parentheses. Given disjoint sets $\mathcal{F}$ and $\mathcal{V}$, whose elements we call function symbols and variables, respectively, the set $\mathfrak{T}$ of pre-terms over $\mathcal{F}$ and $\mathcal{V}$ is generated by the grammar $\mathfrak{T} ::= \mathcal{F} \mid \mathcal{V} \mid (\mathfrak{T}\ \mathfrak{T})$. Left-associativity is assigned to the juxtaposition operation, called application, so for instance $t_0\ t_1\ t_2$ stands for $((t_0\ t_1)\ t_2)$.

We assume every function symbol and variable is assigned a unique type. Typing works as expected: if pre-terms t_0 and t_1 have types $A \to B$ and A, respectively, $t_0\ t_1$ has type B. The set $T(\mathcal{F}, \mathcal{V})$ of terms over $\mathcal{F}$ and $\mathcal{V}$ consists of pre-terms that have a type. We write $t : A$ if a term t has type A. We assume there are infinitely many variables of each type.

The set $\mathrm{Var}(t)$ of variables in $t \in T(\mathcal{F}, \mathcal{V})$ is defined by $\mathrm{Var}(\mathsf{f}) = \emptyset$ for $\mathsf{f} \in \mathcal{F}$, $\mathrm{Var}(x) = \{\, x \,\}$ for $x \in \mathcal{V}$ and $\mathrm{Var}(t_0\ t_1) = \mathrm{Var}(t_0) \cup \mathrm{Var}(t_1)$. A term t is called ground if $\mathrm{Var}(t) = \emptyset$.

For constrained rewriting, we make further assumptions. First, we assume there is a distinguished subset $\mathcal{S}_\vartheta$ of $\mathcal{S}$, called the set of theory sorts. The grammar $\mathcal{T}_\vartheta ::= \mathcal{S}_\vartheta \mid (\mathcal{S}_\vartheta \to \mathcal{T}_\vartheta)$ generates the set $\mathcal{T}_\vartheta$ of theory types over $\mathcal{S}_\vartheta$. Note that a theory type is essentially a non-empty list of theory sorts. Next, we assume there is a distinguished subset $\mathcal{F}_\vartheta$ of $\mathcal{F}$, called the set of theory symbols, and the type

Date: doi:10.4230/LIPIcs.FSCD.2025.20; CC BY4.0.

of every theory symbol is in $\mathcal{T}_\vartheta$, which means the type of any argument passed to a theory symbol is a theory sort. Theory symbols whose type is a theory sort are called theory values.[222]Such theory symbols are simply called values in [22]. In this paper, we call them differently so that they are not to be confused with term values as in call by value. Elements of $T(\mathcal{F}_\vartheta, \mathcal{V})$ are called theory terms.

Theory symbols are interpreted in an underlying theory: given an $\mathcal{S}_\vartheta$-indexed family of sets $(\mathfrak{X}_A)_{A\in\mathcal{S}_\vartheta}$, we extend it to a $\mathcal{T}_\vartheta$-indexed family by letting $\mathfrak{X}_{A\to B}$ be the set of mappings from $\mathfrak{X}_A$ to $\mathfrak{X}_B$; an interpretation of theory symbols is a $\mathcal{T}_\vartheta$-indexed family of mappings $(\llbracket\cdot\rrbracket_A)_{A\in\mathcal{T}_\vartheta}$ where $\llbracket\cdot\rrbracket_A$ assigns to each theory symbol of type A an element of $\mathfrak{X}_A$ and is bijective if $A \in \mathcal{S}_\vartheta$. Given an interpretation of theory symbols $(\llbracket\cdot\rrbracket_A)_{A\in\mathcal{T}_\vartheta}$, we extend each indexed mapping $\llbracket\cdot\rrbracket_B$ to one that assigns to each ground theory term of type B an element of $\mathfrak{X}_B$ by letting $\llbracket t_0\ t_1\rrbracket_B$ be $\llbracket t_0\rrbracket_{A\to B}(\llbracket t_1\rrbracket_A)$. We write just $\llbracket\cdot\rrbracket$ when the type can be deduced.

Example 1.

Let $\mathcal{S}_\vartheta$ be $\{\,\mathsf{int}\,\}$. Then $\mathsf{int}\to\mathsf{int}\to\mathsf{int}$ is a theory type over $\mathcal{S}_\vartheta$ while $(\mathsf{int}\to\mathsf{int})\to\mathsf{int}$ is not. Let $\mathcal{F}_\vartheta$ be $\{-\}\cup\mathbb{Z}$ where $-:\mathsf{int}\to\mathsf{int}\to\mathsf{int}$ and $n:\mathsf{int}$ for all $n\in\mathbb{Z}$. The theory values are the elements of $\mathbb{Z}$. Let $\mathfrak{X}_{\mathsf{int}}$ be $\mathbb{Z}$, $\llbracket\cdot\rrbracket_{\mathsf{int}}$ be the identity mapping and $\llbracket-\rrbracket$ be the mapping $\lambda m.\,\lambda n.\,m-n$. The interpretation of $(-)\ 1$ is the mapping $\lambda n.\,1-n$ (note that functions in our setting are curried).

Type-preserving mappings from $\mathcal{V}$ to $T(\mathcal{F},\mathcal{V})$ are called substitutions. The domain of a substitution σ is the set $\mathrm{dom}(\sigma)=\{\,x\in\mathcal{V}\mid\sigma(x)\neq x\,\}$. Let $[x_1:=t_1,\dots,x_n:=t_n]$ denote the substitution σ such that $\mathrm{dom}(\sigma)\subseteq\{\,x_1,\dots,x_n\,\}$ and $\sigma(x_i)=t_i$ for all i. Every substitution σ extends to a type-preserving mapping $\bar\sigma$ from $T(\mathcal{F},\mathcal{V})$ to $T(\mathcal{F},\mathcal{V})$. We write $t\sigma$ for $\bar\sigma(t)$ and define it as follows: $\mathsf{f}\sigma=\mathsf{f}$ for $\mathsf{f}\in\mathcal{F}$, $x\sigma=\sigma(x)$ for $x\in\mathcal{V}$ and $(t_0\ t_1)\sigma=(t_0\sigma)\ (t_1\sigma)$.

A context is a term containing a hole. That is, if we let $\Box$ be a special symbol and assign to it a type A, a context $C[]$ is an element of $T(\mathcal{F},\mathcal{V}\cup\{\,\Box\,\})$ such that $\Box$ occurs in $C[]$ exactly once. Given a term $t:A$, let $C[t]$ denote the term produced by replacing $\Box$ in $C[]$ with t.

A term t is called a (maximally applied) subterm of a term s, written as $s\trianglerighteq t$, if either $s=t$, $s=s_0\ s_1$ where $s_1\trianglerighteq t$, or $s=s_0\ s_1$ where $s_0\trianglerighteq t$ and $s_0\neq t$; i.e., $s=C[t]$ for $C[]$ that is not of form $C'[\Box\ t_1]$. We write $s\triangleright t$ and call t a proper subterm of s if $s\trianglerighteq t$ and $s\neq t$.

Constrained Rewriting.

Constrained rewriting requires the theory sort bool: we henceforth assume that $\mathsf{bool}\in\mathcal{S}_\vartheta$, $\{\mathsf{f},\mathsf{t}\}\subseteq\mathcal{F}_\vartheta$, $\mathfrak{X}_{\mathsf{bool}}=\{\,0,1\,\}$, $\llbracket\mathsf{f}\rrbracket_{\mathsf{bool}}=0$ and $\llbracket\mathsf{t}\rrbracket_{\mathsf{bool}}=1$. Moreover, we require that $\mathcal{F}_\vartheta$ includes symbols $\wedge:\mathsf{bool}\to\mathsf{bool}\to\mathsf{bool}$, and for each sort $A\in\mathcal{S}_\vartheta$ also a symbol $\equiv_A\colon A\to A\to\mathsf{bool}$, interpreted respectively as conjunction and equality operators.

A logical constraint is a theory term φ such that φ has type bool and the type of each variable in $\mathrm{Var}(\varphi)$ is a theory sort. A (constrained) rewrite rule is a triple $\ell\to r\ [\varphi]$ where ℓ and r are terms which have the same type, φ is a logical constraint, $\mathrm{Var}(r)\setminus\mathrm{Var}(\ell)\subseteq\mathrm{Var}(\varphi)$, and ℓ is a pattern that takes the form $\mathsf{f}\ t_1\cdots t_n$ for some $\mathsf{f}\in\mathcal{F}$ and contains at least one function symbol in $\mathcal{F}\setminus\mathcal{F}_\vartheta$. Here a pattern is a term whose subterms are either $\mathsf{f}\ t_1\cdots t_n$ for some $\mathsf{f}\in\mathcal{F}$ or a variable.[333]As usual in term rewriting, we do not require that each variable occurs at most once in a pattern. A substitution σ is said to respect a logical constraint φ if $\sigma(x)$ is a theory value for all $x\in\mathrm{Var}(\varphi)$ and $\llbracket\varphi\sigma\rrbracket=1$; σ respects the rule $\ell\to r\ [\varphi]$ if it respects φ.

A logically constrained simply-typed term rewriting system (LCSTRS) collects the above data – $\mathcal{S}$, $\mathcal{S}_\vartheta$, $\mathcal{F}$, $\mathcal{F}_\vartheta$, $\mathcal{V}$, $(\mathfrak{X}_A)$ and $\llbracket\cdot\rrbracket$ – along with a set $\mathcal{R}$ of rewrite rules. We usually let $\mathcal{R}$ alone stand for the system. The set $\mathcal{R}$ induces the rewrite

relation $\to_{\mathcal{R}}$ over terms: $t \to_{\mathcal{R}} t'$ if and only if there exists a context $C[]$ such that one of the following holds:
(1)
$t = C[\ell\sigma]$ and $t' = C[r\sigma]$ for some $\ell \to r\ [\varphi] \in \mathcal{R}$ and substitution σ which respects φ, or
(2)
$t = C[\mathsf{f}\ v_1 \cdots v_n]$ and $t' = C[v']$ for some theory symbol f and some theory values $v_1, \ldots, v_n, v'$ with $n > 0$ and $[\![\mathsf{f}\ v_1 \cdots v_n]\!] = [\![v']\!]$.

If $t \to_{\mathcal{R}} t'$ due to the second condition, we also write $t \to_\kappa t'$ and call it a calculation step. Theory symbols that are not a theory value are called calculation symbols. Let $t{\downarrow_\kappa}$ denote the (unique) κ-normal form of t, i.e., the term t' such that $t \to_\kappa^* t'$ and $t' \not\to_\kappa t''$ for any t''. For example, $(\mathsf{f}\ (7 * (3 * 2))){\downarrow_\kappa} = \mathsf{f}\ 42$ if f is not a calculation symbol, or if $\mathsf{f} : \mathsf{int} \to A \to B$.

A term t is in normal form if there is no term t' such that $t \to_{\mathcal{R}} t'$. Given an LCSTRS $\mathcal{R}$, the set $\mathcal{NF}(\mathcal{R})$ contains all terms that are in normal form with respect to $\to_{\mathcal{R}}$.

Given an LCSTRS $\mathcal{R}$, f is called a defined symbol if there is at least one rewrite rule of the form $\mathsf{f}\ t_1 \cdots t_n \to r\ [\varphi]$. Let $\mathcal{D}$ denote the set of defined symbols. Theory values and function symbols in $\mathcal{F} \setminus (\mathcal{F}_\vartheta \cup \mathcal{D})$ are called constructors.

Example 2.
Consider the following LCSTRS $\mathcal{R}$, where $\mathsf{gcdlist} : \mathsf{intlist} \to \mathsf{int}$, $\mathsf{fold} : (\mathsf{int} \to \mathsf{int} \to \mathsf{int}) \to \mathsf{int} \to \mathsf{intlist} \to \mathsf{int}$ and $\mathsf{gcd} : \mathsf{int} \to \mathsf{int} \to \mathsf{int}$:

$$\mathsf{gcdlist} \to \mathsf{fold}\ \mathsf{gcd}\ 0 \qquad \mathsf{fold}\ f\ y\ \mathsf{nil} \to y \qquad \mathsf{fold}\ f\ y\ (\mathsf{cons}\ x\ l) \to f\ x\ (\mathsf{fold}\ f\ y\ l)$$
$$\mathsf{gcd}\ m\ n \to \mathsf{gcd}\ (-m)\ n \quad [m < 0] \qquad \mathsf{gcd}\ m\ n \to \mathsf{gcd}\ m\ (-n) \quad [n < 0]$$
$$\mathsf{gcd}\ m\ 0 \to m \quad [m \geq 0] \qquad \mathsf{gcd}\ m\ n \to \mathsf{gcd}\ n\ (m \bmod n) \quad [m \geq 0 \wedge n > 0]$$

We use infix notation for some binary operators, and omit logical constraints that are t. Here is a rewrite sequence: $\mathsf{gcdlist}\ (\mathsf{cons}\ (1+1)\ \mathsf{nil}) \to_{\mathcal{R}} \mathsf{fold}\ \mathsf{gcd}\ 0\ (\mathsf{cons}\ (1+1)\ \mathsf{nil}) \to_{\mathcal{R}} \mathsf{gcd}\ (1+1)\ (\mathsf{fold}\ \mathsf{gcd}\ 0\ \mathsf{nil}) \to_{\mathcal{R}} \mathsf{gcd}\ (1+1)\ 0 \to_\kappa \mathsf{gcd}\ 2\ 0 \to_{\mathcal{R}} 2$.

Innermost Rewriting.
The innermost rewrite relation $\xrightarrow{\mathrm{i}}_{\mathcal{R}}$ requires all the proper subterms of a redex to be in normal form: $t \xrightarrow{\mathrm{i}}_{\mathcal{R}} t'$ if and only if either (1) there exist a context $C[]$, a substitution σ and a rewrite rule $\ell \to r\ [\varphi] \in \mathcal{R}$ such that $t = C[\ell\sigma]$, $s \not\to_{\mathcal{R}} s'$ for any proper subterm s of $\ell\sigma$ and any s', $t' = C[r\sigma]$ and σ respects φ, or (2) $t \to_\kappa t'$.

Similarly, for an LCSTRS $\mathcal{Q}$ we define the relation $\xrightarrow{\mathcal{Q}}_{\mathcal{R}}$ as reduction using $\to_{\mathcal{R}}$ where all proper subterms of a redex are in $\to_{\mathcal{Q}}$-normal form: $t \xrightarrow{\mathcal{Q}}_{\mathcal{R}} t'$ if and only if either (1) there exist $C[]$, σ and $\ell \to r\ [\varphi] \in \mathcal{R}$ such that $t = C[\ell\sigma]$, $s \not\to_{\mathcal{Q}} s'$ for any proper subterm s of $\ell\sigma$ and any s', $t' = C[r\sigma]$ and σ respects φ, or (2) $t \to_\kappa t'$. Note that $\xrightarrow{\mathrm{i}}_{\mathcal{R}}$ is the same as $\xrightarrow{\mathcal{R}}_{\mathcal{R}}$.

Call-By-Value Rewriting.
We may further restrict a redex by requiring each proper subterm to be a ground term value. Here a term value is either a variable or a term $\mathsf{f}\ v_1 \cdots v_n$ where f is a function symbol, v_i is a term value for all i, and (1) if there is a rule $\mathsf{f}\ t_1 \cdots t_k \to r\ [\varphi] \in \mathcal{R}$, then $k > n$; (2) if f is a calculation symbol, then it takes at least $n+1$ arguments (i.e., $\mathsf{f} : A_1 \to \cdots \to A_{n+1} \to B$). In this paper, we will typically refer to term values as just values. By definition, all theory values are values, and all values are in normal form.

The definition of the call-by-value rewrite relation $\xrightarrow{\mathrm{v}}_{\mathcal{R}}$ follows the pattern of $\xrightarrow{\mathrm{i}}_{\mathcal{R}}$: $t \xrightarrow{\mathrm{v}}_{\mathcal{R}} t'$ if and only if either (1) there exist a context $C[]$, a substitution σ

and a rewrite rule $\ell \to r\ [\varphi] \in \mathcal{R}$ such that $t = C[\ell\sigma]$, s is a ground value for each proper subterm s of $\ell\sigma$, $t' = C[r\sigma]$ and σ respects φ, or (2) $t \to_\kappa t'$.

Example 3.

The rewrite sequence in Example 2 is not innermost (and therefore not call-by-value). An example of a call-by-value rewrite sequence is:

$$\mathsf{gcdlist}\ (\mathsf{cons}\ (1+1)\ \mathsf{nil}) \xrightarrow{\mathrm{v}}_{\mathcal{R}} \mathsf{fold}\ \mathsf{gcd}\ 0\ (\mathsf{cons}\ (1+1)\ \mathsf{nil}) \to_\kappa \mathsf{fold}\ \mathsf{gcd}\ 0\ (\mathsf{cons}\ 2\ \mathsf{nil}) \xrightarrow{\mathrm{v}}_{\mathcal{R}} \mathsf{gcd}\ 2\ (\mathsf{fold}\ \mathsf{gcd}\ 0\ \mathsf{nil}) \xrightarrow{\mathrm{v}}_{\mathcal{R}} \mathsf{gcd}\ 2\ 0 \xrightarrow{\mathrm{v}}_{\mathcal{R}} 2.$$

Note that the redex in the first step is $\mathsf{gcdlist}$, which does not have $1 + 1$ as subterm.

Example 4.

Consider the LCSTRS $\mathcal{R} = \{\mathsf{hd}\ (\mathsf{cons}\ x\ l) \to x,\ \mathsf{tl}\ (\mathsf{cons}\ x\ l) \to l\}$. Then the rewrite step $\mathsf{hd}\ (\mathsf{cons}\ 42\ (\mathsf{tl}\ \mathsf{nil})) \xrightarrow{\mathrm{i}}_{\mathcal{R}} 42$ is an innermost step, but not a call-by-value step. The reason is that the subterm $\mathsf{tl}\ \mathsf{nil}$ of the redex is in normal form, but not a value: innermost rewriting also allows for rewriting above function calls that are not defined on the given arguments; in a language like OCaml, the computation would abort with an error.

2.2 Accessibility and Computability

Accessibility.

Assume given a sort ordering – a quasi-ordering $\succsim$ over $\mathcal{S}$ whose strict part $\succ = \succsim \setminus \precsim$ is well-founded. We inductively define two relations $\succsim_+$ and $\succ_-$ over $\mathcal{S}$ and $\mathcal{T}$: for a sort A and a type $B = B_1 \to \cdots \to B_n \to C$ where C is a sort and $n \geq 0$, $A \succsim_+ B$ if $A \succsim C$ and $A \succ_- B_i$ for all i, and $A \succ_- B$ if $A \succ C$ and $A \succsim_+ B_i$ for all i.

Given a function symbol $\mathsf{f} : A_1 \to \cdots \to A_n \to B$ where B is a sort, the set $\mathrm{Acc}(\mathsf{f})$ of the accessible argument positions of f is defined as $\{\, i \mid B \succsim_+ A_i \,\}$. A term t is called an accessible subterm of a term s, written as $s \trianglerighteq_{\mathrm{acc}} t$, if either $s = t$, or $s = \mathsf{f}\ s_1 \cdots s_n$ for some $\mathsf{f} \in \mathcal{F}$ and there exists $k \in \mathrm{Acc}(\mathsf{f})$ such that $s_k \trianglerighteq_{\mathrm{acc}} t$. An LCSTRS $\mathcal{R}$ is called accessible function passing (AFP) if there exists a sort ordering such that for all $\mathsf{f}\ s_1 \cdots s_n \to r\ [\varphi] \in \mathcal{R}$ where $\mathsf{f} \in \mathcal{F}$ and $x \in \mathrm{Var}(r) \setminus \mathrm{Var}(\varphi)$, there exists k such that $s_k \trianglerighteq_{\mathrm{acc}} x$.

Example 5.

An LCSTRS $\mathcal{R}$ is AFP (with $\succsim$ equating all the sorts) if for all $\mathsf{f}\ s_1 \cdots s_n \to r\ [\varphi] \in \mathcal{R}$ where $\mathsf{f} \in \mathcal{F}$ and $i \in \{\, 1, \ldots, n \,\}$, the type of each proper subterm of s_i is a sort. This is typical for many common examples of higher-order programs manipulating first-order data; e.g., integer recursion or list folding, mapping or filtering; hence, Example 2 is AFP.

Consider $\{\, \mathsf{complst}\ \mathsf{fnil}\ x \to x, \mathsf{complst}\ (\mathsf{fcons}\ f\ l)\ x \to \mathsf{complst}\ l\ (f\ x) \,\}$, where $\mathsf{complst} : \mathsf{funlist} \to \mathsf{int} \to \mathsf{int}$ composes a list of functions. This system is AFP with $\mathsf{funlist} \succ \mathsf{int}$.

Consider $\{\, \mathsf{app}\ (\mathsf{lam}\ f) \to f \,\}$ where $\mathsf{app} : \mathsf{o} \to \mathsf{o} \to \mathsf{o}$ and $\mathsf{lam} : (\mathsf{o} \to \mathsf{o}) \to \mathsf{o}$. This system encodes the untyped lambda-calculus, with app serving as the application symbol and lam as a wrapper for abstractions. It is not AFP since $\mathsf{o} \succ \mathsf{o}$ cannot be true.

Computability.

A term is called neutral if it takes the form $x\ t_1 \cdots t_n$ for some variable x. A set of reducibility candidates, or an RC-set, for a type-preserving relation $\to$ over terms – which may stand for $\to_{\mathcal{R}}$, $\xrightarrow{\mathrm{i}}_{\mathcal{R}}$, but also $\xrightarrow{\mathrm{v}}_{\mathcal{R}}$ – is an $\mathcal{S}$-indexed family of sets $(I_A)_{A \in \mathcal{S}}$ (let I denote $\bigcup_A I_A$) satisfying the following conditions:

(1)

Each element of I_A is a terminating (with respect to $\to$) term of type A.

(2)

Given terms s and t such that $s \to t$, if s is in I_A, so is t.

(3)

Given a neutral term s, if t is in I_A for all t such that $s \to t$, so is s.

A term t_0 is called I-computable if either the type of t_0 is a sort and $t_0 \in I$, or the type of t_0 is $A \to B$ and $t_0\ t_1$ is I-computable for all I-computable $t_1 : A$.

We are interested in a specific RC-set $\mathbb{C}$:

Theorem 6 (see [13]).

Given a sort ordering and an RC-set I for $\to$, let $\Rrightarrow_I$ be the relation over terms such that $s \Rrightarrow_I t$ if and only if both s and t have a base type, $s = \mathsf{f}\ s_1 \cdots s_m$ for some function symbol f, $t = s_k\ t_1 \cdots t_n$ for some $k \in \mathrm{Acc}(\mathsf{f})$ and t_i is I-computable for all i.

Given an LCSTRS $\mathcal{R}$ with a sort ordering, there exists an RC-set $\mathbb{C}$ for $\to$ such that $t \in \mathbb{C}_A$ if and only if $t : A$ is terminating with respect to $\to \cup \Rrightarrow_{\mathbb{C}}$, and for all t' such that $t \to^* t'$, if $t' = \mathsf{f}\ t_1 \cdots t_n$ for some function symbol f, t_i is $\mathbb{C}$-computable for all $i \in \mathrm{Acc}(\mathsf{f})$.

$\mathbb{C}$-computability with respect to the innermost reduction relation $\xrightarrow{\mathrm{i}}_{\mathcal{R}}$ is at the heart of the soundness proofs for the DP framework defined in this paper (Theorems 14 and 36).

3 The Transformation of Call-By-Value Systems

Let us reflect on the shape of call-by-value LCSTRSs. First, we observe that if a pattern t is not a value, neither are its instances, i.e., $t\sigma$ is not a value for any substitution σ. Hence, under call by value, a rewrite rule $\ell \to r\ [\varphi]$ where not all the proper subterms of ℓ are values is never applicable, and we can exclude such rewrite rules without loss of generality.

Second, programming languages hardly allow the addition of constructors to predefined (in particular, primitive) data types, such as integers. Given an LCSTRS, in practice, we would also expect, for example, that integer literals are the only constructors for int. Formally, we call a theory sort B inextensible if every function symbol $\mathsf{f} : A_1 \to \cdots \to A_n \to B$ is a theory symbol or a defined symbol. We do not require that all theory sorts are inextensible; rather, we assume that for each rewrite rule $\ell \to r\ [\varphi]$, all the variables in $\mathrm{Var}(\ell)$ whose type is an inextensible theory sort are also in $\mathrm{Var}(\varphi)$. We can do so without loss of generality because such a variable can only be instantiated to a theory value under call by value. We only consider inextensible theory sorts in examples below.

We write $\varphi, x_1, \ldots, x_n$ for $\varphi \wedge x_1 \equiv x_1 \wedge \cdots \wedge x_n \equiv x_n$, and conclude the above discussion:

Lemma 7.

We apply the below transformation to an LCSTRS $\mathcal{R}$ and let $\mathcal{R}'$ be the outcome.

(1)

Remove all $\ell \to r\ [\varphi]$ where ℓ has a proper subterm that is not a value.

(2)

Replace all remaining $\ell \to r\ [\varphi]$ with $\ell \to r\ [\varphi, x_1, \ldots, x_n]$ where $x_1, \ldots, x_n$ are the variables in $\mathrm{Var}(\ell) \setminus \mathrm{Var}(\varphi)$ whose type is an inextensible theory sort.

Then $t \xrightarrow{\mathrm{v}}_{\mathcal{R}} t'$ if and only if $t \xrightarrow{\mathrm{v}}_{\mathcal{R}'} t'$ for all t and t'.

Example 8.

The LCSTRS $\mathcal{R}$ from Example 2 is transformed as follows: (1) no rewrite rules are removed, and (2) four rewrite rules are replaced by the ones below.

$\mathsf{fold}\ f\ y\ \mathsf{nil} \to y\ [\mathsf{t}, y]$ $\mathsf{gcd}\ m\ n \to \mathsf{gcd}\ (-m)\ n\ [m < 0, n]$ $\mathsf{fold}\ f\ y\ (\mathsf{cons}\ x\ l) \to f\ x\ (\mathsf{fold}\ f\ y\ l)\ [\mathsf{t}, x, y]$ $\mathsf{gcd}\ m\ n \to \mathsf{gcd}\ m\ (-n)\ [n < 0, m]$ We will refer to the result of the transformation as $\mathcal{R}_{\mathsf{gcd}}$.

We are particularly interested in call-by-value termination. However, innermost termination is more commonly considered in the term rewriting community, and has been studied extensively for first-order term rewriting. In practice, its difference from call-by-value termination is often irrelevant, and their respective techniques are generally similar. Since all values are in normal form, $\xrightarrow{\mathrm{v}}_{\mathcal{R}}$ is terminating if $\xrightarrow{\mathrm{i}}_{\mathcal{R}'}$ is. Hence, we shall formulate our results in terms of innermost rewriting, for the sake of more generality and less bookkeeping.

4 Static Dependency Pairs and Chains

The dependency pair method [3] analyzes the recursive structure of function calls. Its variants are at the heart of most modern automatic termination analyzers for various styles of term rewriting. Static dependency pairs [35, 13], a higher-order generalization of the method, are adapted to LCSTRSs in [21]. Now we extend this adaptation for the evaluation strategies.

First, we recall a notation:

Definition 9.

Given an LCSTRS $\mathcal{R}$, let $\mathcal{F}^\sharp$ be $\mathcal{F} \uplus \{ \mathsf{f}^\sharp \mid \mathsf{f} \in \mathcal{D} \}$ where $\mathcal{D}$ is the set of defined symbols in $\mathcal{R}$ and $\mathsf{f}^\sharp$ is a fresh function symbol for all f. Let dp be a fresh sort, and for each defined symbol $\mathsf{f} : A_1 \to \cdots \to A_n \to B$ where $B \in \mathcal{S}$, we assign $\mathsf{f}^\sharp : A_1 \to \cdots \to A_n \to \mathsf{dp}$. Given a term $t = \mathsf{f}\ t_1 \cdots t_n \in T(\mathcal{F}, \mathcal{V})$ where $\mathsf{f} \in \mathcal{D}$, let $t^\sharp$ denote $\mathsf{f}^\sharp\ t_1 \cdots t_n \in T(\mathcal{F}^\sharp, \mathcal{V})$.

The definition of a static dependency pair is adapted as follows:

Definition 10.

A static dependency pair (SDP) is a triple $s^\sharp \Rightarrow t^\sharp\ [\varphi]$ where $s^\sharp$ and $t^\sharp$ are terms of type dp, and φ is a logical constraint. This SDP is considered call-by-value if all the proper subterms of $s^\sharp$ are values. For a rewrite rule $\ell \to r\ [\varphi]$, let $\mathrm{SDP}(\ell \to r\ [\varphi])$ denote the set of SDPs of form $\ell^\sharp\ x_1 \cdots x_m \Rightarrow \mathsf{g}^\sharp\ t_1 \cdots t_q\ y_{q+1} \cdots y_n\ [\varphi]$ such that

(1)
$\ell^\sharp : A_1 \to \cdots \to A_m \to \mathsf{dp}$ with fresh variables $x_1, \ldots, x_m$,

(2)
$r\ x_1 \cdots x_m \trianglerighteq \mathsf{g}\ t_1 \cdots t_q$ for $\mathsf{g} \in \mathcal{D}$, and

(3)
$\mathsf{g}^\sharp : B_1 \to \cdots \to B_n \to \mathsf{dp}$ with fresh variables $y_{q+1}, \ldots, y_n$.

Let $\mathrm{SDP}(\mathcal{R})$ be $\bigcup_{\ell \to r\ [\varphi] \in \mathcal{R}} \mathrm{SDP}(\ell \to r\ [\varphi])$. For $s^\sharp \Rightarrow t^\sharp\ [\mathrm{t}]$, we just write $s^\sharp \Rightarrow t^\sharp$.

Compared to the quadruple definition in [21], this definition omits the L component, because here its bookkeeping role can be assumed by the logical constraint.

Intuitively, every dependency pair in $\mathrm{SDP}(\mathcal{R})$ represents a function call in $\mathcal{R}$. If $\mathcal{R}$ is non-terminating, there must be an infinite "chain" of function calls whose arguments are terminating (in other words, the chain is "minimal": all the proper subterms are terminating). To distinguish the head symbols of those minimal potentially non-terminating terms, we write $\mathsf{f}^\sharp$ instead of f at head positions in SDPs. Then calls to a defined symbol f (the original) can be assumed to terminate, which is shown separately via $\mathsf{f}^\sharp$. And the non-existence of an infinite chain starting from a call to any $\mathsf{f}^\sharp$ in $\mathrm{SDP}(\mathcal{R})$ implies the termination of $\mathcal{R}$.

Example 11.

Following Example 8, $\mathrm{SDP}(\mathcal{R}_{\mathsf{gcd}})$ consists of the following SDPs:

(1)
$\mathsf{gcdlist}^\sharp\ l' \Rightarrow \mathsf{gcd}^\sharp\ m'\ n'$

(2)
$\mathsf{gcdlist}^\sharp\ l' \Rightarrow \mathsf{fold}^\sharp\ \mathsf{gcd}\ 0\ l'$

(3)
$\mathsf{gcd}^\sharp\ m\ n \Rightarrow \mathsf{gcd}^\sharp\ (-m)\ n\ [m < 0, n]$
(4)
$\mathsf{gcd}^\sharp\ m\ n \Rightarrow \mathsf{gcd}^\sharp\ m\ (-n)\ [n < 0, m]$
(5)
$\mathsf{gcd}^\sharp\ m\ n \Rightarrow \mathsf{gcd}^\sharp\ n\ (m \bmod n)\ [m \geq 0 \wedge n > 0]$
(6)
$\mathsf{fold}^\sharp\ f\ y\ (\mathsf{cons}\ x\ l) \Rightarrow \mathsf{fold}^\sharp\ f\ y\ l\ [\mathsf{t}, x, y]$

In this paper, a set $\mathcal{R}$ of rewrite rules plays two roles: (1) it specifies how to rewrite arguments in SDPs, and (2) it determines which rewrite steps are innermost and which terms are values. In Sections 5 and 6, a given set $\mathcal{R}$ in its first role may be modified (with rewrite rules removed). In this process, if we do not keep the original set, there can be unintended consequences for its second role. For example, consider $\mathcal{R} = \{\, \mathsf{f}\ x \to r_1, \mathsf{a} \to r_2 \,\}$. Note that the first rewrite rule is not applicable to $\mathsf{f}\ \mathsf{a}$ with respect to innermost rewriting. Now if we remove the second rewrite rule from $\mathcal{R}$ and consider the new set, the remaining rewrite rule will be applicable, and the term a will be a value, which is generally an unwanted change.

Hence, we keep a copy of the original set $\mathcal{R}$ of rewrite rules to faithfully determine which rewrite steps are innermost and which terms are values. Following [18, 41], we let $\mathcal{Q}$ denote this copy. In contrast to the literature, we consider $\mathcal{Q}$ fixed throughout the analysis of a given system.444A fixed set $\mathcal{Q}$ suffices in this paper because our DP processors may remove rewrite rules, but may not add new rewrite rules or change the signature (e.g., with semantic labeling [45]). Now we formalize the idea of a chain of function calls in the innermost setting:

Definition 12.
Given a set $\mathcal{P}$ of SDPs and a set $\mathcal{R}$ of rewrite rules, an innermost $(\mathcal{P}, \mathcal{R})$-chain is a (finite or infinite) sequence $(s_0^\sharp \Rightarrow t_0^\sharp\ [\varphi_0], \sigma_0), (s_1^\sharp \Rightarrow t_1^\sharp\ [\varphi_1], \sigma_1), \ldots$ such that for all i, $s_i^\sharp \Rightarrow t_i^\sharp\ [\varphi_i] \in \mathcal{P}$, σ_i is a substitution which respects φ_i, $s_i^\sharp\sigma_i$ is in normal form with respect to $\to_{\mathcal{Q}}$, and $t_{i-1}^\sharp\sigma_{i-1} \xrightarrow{\mathcal{Q}}_{\mathcal{R}}^* s_i^\sharp\sigma_i$ if $i > 0$.

Example 13.
Following Example 11, $(1, [l' := \mathsf{nil}, m' := 24, n' := 18]), (5, [m := 24, n := 18]), (5, [m := 18, n := 6])$ is an innermost $(\mathrm{SDP}(\mathcal{R}_{\mathsf{gcd}}), \mathcal{R}_{\mathsf{gcd}})$-chain.

In the above definition, the requirement that $s_i^\sharp\sigma_i$ is in normal form implies that $\sigma_i(x)$ is in normal form for all $x \in \mathrm{Var}(s_i^\sharp)$, including fresh variables such as l' in SDP (1). Hence, the call-by-value (and therefore innermost) rewrite sequence in Example 3 does not directly translate to an innermost $(\mathrm{SDP}(\mathcal{R}), \mathcal{R})$-chain. Nevertheless, we have the following result:

Proof Idea.
The full proof is in Section A.2, and is very similar to its counterpart for termination with respect to full rewriting [21]: in a non-terminating LCSTRS, we can always identify a non-terminating base-type term $t = \mathsf{f}\ t_1 \cdots t_n$ such that all t_i are $\mathbb{C}$-computable; and from an infinite reduction $t \xrightarrow{\mathrm{i}}_{\mathcal{R}} t' \xrightarrow{\mathrm{i}}_{\mathcal{R}} t'' \xrightarrow{\mathrm{i}}_{\mathcal{R}} \ldots$ we can find s, u such that $s = \mathsf{f}\ (t_1 \downarrow_{\mathcal{R}}) \cdots (t_n \downarrow_{\mathcal{R}})$, $(s^\sharp, u^\sharp)$ is an instance of a dependency pair, and u is again non-terminating with $\mathbb{C}$-computable arguments. The primary difference from [21] is the need to ensure all variables are instantiated to normal forms, which for the fresh variables discussed above means that we must choose the right normal form that still yields non-termination. ◀

5 The Innermost DP Framework

In this section, we modify the constrained DP framework [21] to prove innermost termination. The DP framework is a collection of DP processors, each of which represents a technique.

Definition 15.

A DP problem is a pair $(\mathcal{P}, \mathcal{R})$ where $\mathcal{P}$ is a set of SDPs and $\mathcal{R}$ is a set of rewrite rules. If no infinite innermost $(\mathcal{P}, \mathcal{R})$-chain exists, $(\mathcal{P}, \mathcal{R})$ is called finite. A DP processor is a partial mapping which possibly assigns to a DP problem a set of DP problems. A DP processor ρ is called sound if $(\mathcal{P}, \mathcal{R})$ is finite whenever all the elements of $\rho(\mathcal{P}, \mathcal{R})$ are.

Theorem 14 tells us that an AFP system $\mathcal{R}$ is innermost terminating if $(\mathrm{SDP}(\mathcal{R}), \mathcal{R})$ is a finite DP problem. Given a collection of sound DP processors, we have the following procedure: (1) $\mathcal{Q} \coloneqq \mathcal{R}$, $S \coloneqq \{ (\mathrm{SDP}(\mathcal{R}), \mathcal{R}) \}$; (2) while S contains a DP problem $(\mathcal{P}, \mathcal{R})$ to which some sound DP processor ρ is applicable, $S \coloneqq (S \setminus \{ (\mathcal{P}, \mathcal{R}) \}) \cup \rho(\mathcal{P}, \mathcal{R})$. If this procedure ends with $S = \emptyset$, we can conclude that $\mathcal{R}$ is innermost terminating.

5.1 Graph, Subterm Criterion and Integer Mapping Processors

Three classes of DP processors in [21] remain essentially unchanged in the innermost setting. Let us review their application to the system $\mathcal{R}_{\mathsf{gcd}}$ from Example 8, which is discussed by [21] in the form of Example 2. First, we consider a graph approximation for $(\mathrm{SDP}(\mathcal{R}_{\mathsf{gcd}}), \mathcal{R}_{\mathsf{gcd}})$:

Definition 16.

For a DP problem $(\mathcal{P}, \mathcal{R})$, a graph approximation (G, θ) consists of a finite directed graph G and a mapping θ from $\mathcal{P}$ to the vertices of G such that there is an edge from $\theta(p_0)$ to $\theta(p_1)$ whenever $(p_0, \sigma_0), (p_1, \sigma_1)$ is an innermost $(\mathcal{P}, \mathcal{R})$-chain for some σ_0 and σ_1.

Compared to [21], the main change is that we now check for innermost $(\mathcal{P}, \mathcal{R})$-chains to determine which edges must exist. A graph approximation for $(\mathrm{SDP}(\mathcal{R}_{\mathsf{gcd}}), \mathcal{R}_{\mathsf{gcd}})$ is in Figure 1. Despite the slightly different SDPs (due to the transformation of the call-by-value system), this graph approximation is the same as the one in [21].

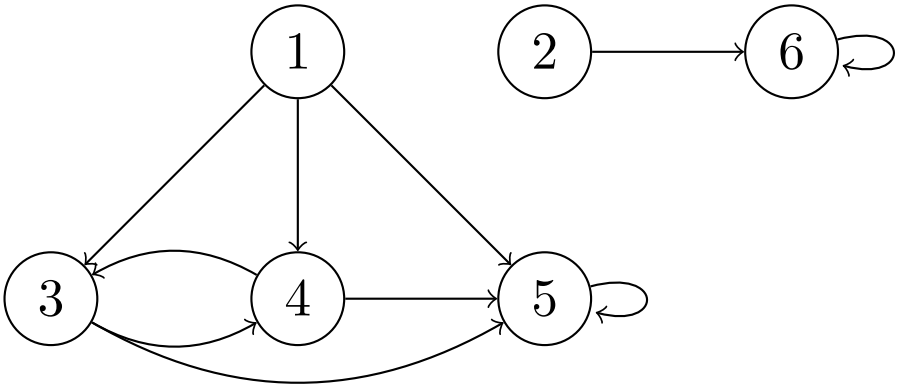


Figure 1: A graph approximation for $(\mathrm{SDP}(\mathcal{R}_{\mathsf{gcd}}), \mathcal{R}_{\mathsf{gcd}})$.

Given a graph approximation, we can decompose the DP problem:

Definition 17.

Given a DP problem $(\mathcal{P}, \mathcal{R})$, a graph processor computes a graph approximation (G, θ) for $(\mathcal{P}, \mathcal{R})$ and the strongly connected components (SCCs) of G, then returns $\{ (\{ p \in \mathcal{P} \mid \theta(p) \text{ belongs to } S \}, \mathcal{R}) \mid S \text{ is a non-trivial SCC of } G \}$.

If a graph processor produces the graph approximation in Figure 1, it will return the set $\{ (\{ 6 \}, \mathcal{R}_{\mathsf{gcd}}), (\{ 3, 4 \}, \mathcal{R}_{\mathsf{gcd}}), (\{ 5 \}, \mathcal{R}_{\mathsf{gcd}}) \}$. Next, we observe that fold is defined by structural recursion, which can be handled by subterm criterion processors.

Let heads($\mathcal{P}$) denote the set of function symbols heading either side of an SDP in $\mathcal{P}$, and we have the following definition:

Definition 18.

A projection ν for a set $\mathcal{P}$ of SDPs is a mapping from heads($\mathcal{P}$) to integers such that $1 \leq \nu(\mathsf{f}^\sharp) \leq n$ if $\mathsf{f}^\sharp : A_1 \to \cdots \to A_n \to \mathsf{dp}$. Let $\bar{\nu}(\mathsf{f}^\sharp\ t_1 \cdots t_n)$ denote $t_{\nu(\mathsf{f}^\sharp)}$. A projection ν is said to $\rhd$-orient a subset $\mathcal{P}'$ of $\mathcal{P}$ if $\bar{\nu}(s^\sharp) \rhd \bar{\nu}(t^\sharp)$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P}'$ and $\bar{\nu}(s^\sharp) = \bar{\nu}(t^\sharp)$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P} \setminus \mathcal{P}'$. A subterm criterion processor maps $(\mathcal{P}, \mathcal{R})$ to $\{\, (\mathcal{P} \setminus \mathcal{P}', \mathcal{R}) \,\}$ for some non-empty $\mathcal{P}' \subseteq \mathcal{P}$ which is $\rhd$-oriented by some projection for $\mathcal{P}$.

Choosing $\nu(\mathsf{fold}^\sharp) = 3$, we have $\bar{\nu}(\mathsf{fold}^\sharp\ f\ y\ (\mathsf{cons}\ x\ l)) = \mathsf{cons}\ x\ l \rhd l = \bar{\nu}(\mathsf{fold}^\sharp\ f\ y\ l)$, so a subterm criterion processor maps $(\{\, 6 \,\}, \mathcal{R}_{\mathsf{gcd}})$ to $\{\, (\emptyset, \mathcal{R}_{\mathsf{gcd}}) \,\}$, and $(\emptyset, \mathcal{R}_{\mathsf{gcd}})$ can (trivially) be removed by a graph processor. Now we are left with $(\{\, 3, 4 \,\}, \mathcal{R}_{\mathsf{gcd}})$ and $(\{\, 5 \,\}, \mathcal{R}_{\mathsf{gcd}})$, which involve recursion over integers. We deal with those by means of integer mappings:

Definition 19.

Given a set $\mathcal{P}$ of SDPs, for all $\mathsf{f}^\sharp \in$ heads($\mathcal{P}$) where $\mathsf{f}^\sharp : A_1 \to \cdots \to A_n \to \mathsf{dp}$, let $\iota(\mathsf{f}^\sharp)$ be the subset of $\{\, 1, \ldots, n \,\}$ such that $i \in \iota(\mathsf{f}^\sharp)$ if and only if $A_i \in \mathcal{S}_\vartheta$ and the i-th argument of any occurrence of $\mathsf{f}^\sharp$ in an SDP $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P}$ is in $T(\mathcal{F}_\vartheta, \mathrm{Var}(\varphi))$. Let $\mathcal{X}(\mathsf{f}^\sharp)$ be a set of fresh variables $\{\, x_i \mid i \in \iota(\mathsf{f}^\sharp) \,\}$ where $x_i : A_i$ for all i. An integer mapping $\mathcal{J}$ for $\mathcal{P}$ is a mapping from heads($\mathcal{P}$) to theory terms such that for all $\mathsf{f}^\sharp$, $\mathcal{J}(\mathsf{f}^\sharp) : \mathsf{int}$ and $\mathrm{Var}(\mathcal{J}(\mathsf{f}^\sharp)) \subseteq \mathcal{X}(\mathsf{f}^\sharp)$. Let $\bar{\mathcal{J}}(\mathsf{f}^\sharp\ t_1 \cdots t_n)$ denote $\mathcal{J}(\mathsf{f}^\sharp)[x_i := t_i]_{i \in \iota(\mathsf{f}^\sharp)}$.

Integer mapping processors handle decreasing integer values:

Definition 20.

Given a set $\mathcal{P}$ of SDPs, an integer mapping $\mathcal{J}$ is said to $>$-orient a subset $\mathcal{P}'$ of $\mathcal{P}$ if $\varphi \models \bar{\mathcal{J}}(s^\sharp) \geq 0 \wedge \bar{\mathcal{J}}(s^\sharp) > \bar{\mathcal{J}}(t^\sharp)$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P}'$, and $\varphi \models \bar{\mathcal{J}}(s^\sharp) \geq \bar{\mathcal{J}}(t^\sharp)$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P} \setminus \mathcal{P}'$, where $\varphi \models \varphi'$ denotes that $[\![\varphi\sigma]\!] = 1$ implies $[\![\varphi'\sigma]\!] = 1$ for each substitution σ which maps variables in $\mathrm{Var}(\varphi) \cup \mathrm{Var}(\varphi')$ to theory values. An integer mapping processor maps $(\mathcal{P}, \mathcal{R})$ to $\{\, (\mathcal{P} \setminus \mathcal{P}', \mathcal{R}) \,\}$ for some non-empty $\mathcal{P}' \subseteq \mathcal{P}$ which is $>$-oriented by some integer mapping for $\mathcal{P}$.

To deal with $(\{\, 5 \,\}, \mathcal{R}_{\mathsf{gcd}})$, let $\mathcal{J}(\mathsf{gcd}^\sharp)$ be x_2 so $\bar{\mathcal{J}}(\mathsf{gcd}^\sharp\ m\ n) = n$, $\bar{\mathcal{J}}(\mathsf{gcd}^\sharp\ n\ (m \bmod n)) = m \bmod n$ and $m \geq 0 \wedge n > 0 \models n \geq 0 \wedge n > m \bmod n$. Then an integer mapping processor returns $\{\, (\emptyset, \mathcal{R}_{\mathsf{gcd}}) \,\}$, and $(\emptyset, \mathcal{R}_{\mathsf{gcd}})$ can (trivially) be removed by a graph processor.

Unlike [21], here an integer mapping processor is applicable to $(\{\, 3, 4 \,\}, \mathcal{R}_{\mathsf{gcd}})$: $\mathcal{J}(\mathsf{gcd}^\sharp) = -x_1$ and the processor returns $\{\, (\{\, 4 \,\}, \mathcal{R}_{\mathsf{gcd}}) \,\}$. Then $(\{\, 4 \,\}, \mathcal{R}_{\mathsf{gcd}})$ can be removed by a graph processor. This simpler proof is due to the transformation of the call-by-value system; in both SDPs, m and n can be instantiated only to theory values, which is not the case in [21], and a theory argument processor is needed there. To elaborate, we consider the system $\mathcal{R}$:

$$\mathsf{f}\ x\ y\ z \to \mathsf{f}\ x\ (x+1)\ (x-1) \quad [y < z] \qquad \mathsf{c}\ x\ y \to x \qquad \mathsf{c}\ x\ y \to y$$

We have $\mathsf{f}\ (\mathsf{c}\ 0\ 3)\ 1\ 2 \to_{\mathcal{R}} \mathsf{f}\ (\mathsf{c}\ 0\ 3)\ ((\mathsf{c}\ 0\ 3) + 1)\ ((\mathsf{c}\ 0\ 3) - 1) \to^+_{\mathcal{R}} \mathsf{f}\ (\mathsf{c}\ 0\ 3)\ 1\ 2$, and therefore $\mathcal{R}$ is not terminating with respect to full rewriting. Under call by value, $\mathcal{R}$ is terminating: the first rewrite rule with x appended to the logical constraint gives the only SDP $\mathsf{f}^\sharp\ x\ y\ z \Rightarrow \mathsf{f}^\sharp\ x\ (x+1)\ (x-1)\ [y < z, x]$, then let $\mathcal{J}(\mathsf{f}^\sharp)$ be $x_3 - x_2$. Here we benefit from call by value.

5.2 The Chaining of Call-By-Value SDPs

Another benefit of call by value is that we may chain together consecutive SDPs, e.g., $\mathsf{fact}^\sharp\ x \Rightarrow \mathsf{u}^\sharp_1\ x\ 1\ [\mathsf{t}, x]$ and $\mathsf{u}^\sharp_1\ x\ z \Rightarrow \mathsf{u}^\sharp_2\ x\ z\ 1\ [\mathsf{t}, x, z]$ may merge to form

$\mathsf{fact}^\sharp\ x \Rightarrow \mathsf{u}_2^\sharp\ x\ 1\ 1\ [\mathsf{t}, x]$. This capability can be important for automatically generated systems.

Definition 21.

Given SDPs $p_0 = (s_0^\sharp \Rightarrow \mathsf{f}^\sharp\ t'_1 \cdots t'_n\ [\varphi_0])$ and $p_1 = (\mathsf{f}^\sharp\ s'_1 \cdots s'_n \Rightarrow t_1^\sharp\ [\varphi_1])$ where $\mathsf{f} \in \mathcal{D}$ and variables are renamed if necessary to avoid name collisions, p_0 and p_1 are called chainable if p_1 is a call-by-value SDP and there exists a substitution σ such that $\mathrm{dom}(\sigma) = \bigcup_{i=1}^n \mathrm{Var}(s'_i)$, $t'_i = s'_i\sigma$ for all i, and $\sigma(x) \in T(\mathcal{F}_\vartheta, \mathrm{Var}(\varphi_0))$ for all $x \in \mathrm{dom}(\sigma) \cap \mathrm{Var}(\varphi_1)$. The chaining $\mathrm{ch}(p_0, p_1)$ of p_0 and p_1 is $s_0^\sharp \Rightarrow t_1^\sharp\sigma\ [\varphi_0 \wedge (\varphi_1\sigma)]$.

Note that in the context of call-by-value rewriting, all SDPs can be assumed to be call-by-value, and therefore it is not particularly restrictive to so require p_1 above. To see the importance of this requirement, consider the DP problem $(\{\, \mathsf{f}^\sharp\ \mathsf{a}\ \mathsf{b} \Rightarrow \mathsf{g}^\sharp\ (\mathsf{h}\ \mathsf{b}\ \mathsf{a}), \mathsf{g}^\sharp\ (\mathsf{h}\ x\ y) \Rightarrow \mathsf{f}^\sharp\ x\ y \,\}, \{\, \mathsf{h}\ \mathsf{b}\ \mathsf{a} \to \mathsf{h}\ \mathsf{a}\ \mathsf{b} \,\})$. If we dropped the call-by-value requirement for the second SDP, the two SDPs would be chainable and yield $\mathsf{f}^\sharp\ \mathsf{a}\ \mathsf{b} \Rightarrow \mathsf{f}^\sharp\ \mathsf{b}\ \mathsf{a}$. However, $\mathsf{f}^\sharp\ \mathsf{b}\ \mathsf{a}$ is not reachable from $\mathsf{f}^\sharp\ \mathsf{a}\ \mathsf{b}$ initially, and replacing the original SDPs with the new one is not sound.

Definition 22.

Given a set $\mathcal{P}$ of SDPs and $\mathsf{f} \in \mathcal{D}$ such that $\mathcal{P}_\mathsf{f}^\ell \neq \emptyset$, $\mathcal{P}_\mathsf{f}^r \neq \emptyset$, $\mathcal{P}_\mathsf{f}^\ell \cap \mathcal{P}_\mathsf{f}^r = \emptyset$ and every pair in $\mathcal{P}_\mathsf{f}^r \times \mathcal{P}_\mathsf{f}^\ell$ is chainable where $\mathcal{P}_\mathsf{f}^\ell = \{\, s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P} \mid s^\sharp = \mathsf{f}^\sharp\ s_1 \cdots s_n \,\}$ and $\mathcal{P}_\mathsf{f}^r = \{\, s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P} \mid t^\sharp = \mathsf{f}^\sharp\ t_1 \cdots t_n \,\}$, a chaining processor assigns to a DP problem $(\mathcal{P}, \mathcal{R})$ the singleton $\{\, ((\mathcal{P} \setminus (\mathcal{P}_\mathsf{f}^\ell \cup \mathcal{P}_\mathsf{f}^r)) \cup \mathcal{P}', \mathcal{R}) \,\}$ where $\mathcal{P}' = \{\, \mathrm{ch}(p_0, p_1) \mid p_0 \in \mathcal{P}_\mathsf{f}^r \text{ and } p_1 \in \mathcal{P}_\mathsf{f}^\ell \,\}$.

Example 23.

Consider the below set $\mathcal{P}$ of SDPs generated from an imperative program [14]:

$\mathsf{fact}^\sharp\ x \Rightarrow \mathsf{u}_1^\sharp\ x\ 1\ [\mathsf{t}, x]\ \mathsf{u}_1^\sharp\ x\ z \Rightarrow \mathsf{u}_2^\sharp\ x\ z\ 1\ [\mathsf{t}, x, z]\ \mathsf{u}_2^\sharp\ x\ z\ i \Rightarrow \mathsf{u}_3^\sharp\ x\ z\ i\ [i \leq x, z]\ \mathsf{u}_3^\sharp\ x\ z\ i \Rightarrow \mathsf{u}_4^\sharp\ x\ (z * i)\ i\ [\mathsf{t}, x, z, i]\ \mathsf{u}_2^\sharp\ x\ z\ i \Rightarrow \mathsf{u}_5^\sharp\ x\ z\ [\neg(i \leq x), z]\ \mathsf{u}_4^\sharp\ x\ z\ i \Rightarrow \mathsf{u}_2^\sharp\ x\ z\ (i+1)\ [\mathsf{t}, x, z, i]$ Chaining processors can iteratively remove the occurrences of $\mathsf{u}_1^\sharp$, $\mathsf{u}_3^\sharp$ and $\mathsf{u}_4^\sharp$, and end with (1) $\mathsf{fact}^\sharp\ x \Rightarrow \mathsf{u}_2^\sharp\ x\ 1\ 1\ [\mathsf{t}, x]$, (2) $\mathsf{u}_2^\sharp\ x\ z\ i \Rightarrow \mathsf{u}_2^\sharp\ x\ (z * i)\ (i+1)\ [i \leq x, z]$, and (3) $\mathsf{u}_2^\sharp\ x\ z\ i \Rightarrow \mathsf{u}_5^\sharp\ x\ z\ [\neg(i \leq x), z]$. Note that a chaining processor for $\mathsf{f} \in \mathcal{D}$ removes all the occurrences of $\mathsf{f}^\sharp$ in $\mathcal{P}$, and therefore the process of iteratively applying chaining processors always terminates (on the assumption that heads$(\mathcal{P})$ is finite). In this example, the above outcome cannot be further chained; there is no chaining processor for u_2 because $\mathcal{P}_{\mathsf{u}_2}^\ell \cap \mathcal{P}_{\mathsf{u}_2}^r$ contains SDP (23).

Example 24.

Consider the following set $\mathcal{P}$ of SDPs:

$\mathsf{f}^\sharp\ x\ y \Rightarrow \mathsf{g}^\sharp\ (\mathsf{comb}\ x\ y)\ [x \geq 0, y]\ \mathsf{g}^\sharp\ (\mathsf{comb}\ z\ w) \Rightarrow \mathsf{h}^\sharp\ (z + w)\ [w \geq 0, z]\ \mathsf{f}^\sharp\ x\ y \Rightarrow \mathsf{g}^\sharp\ (\mathsf{comb}\ (-x)\ y)\ [x < 0, y]\ \mathsf{g}^\sharp\ (\mathsf{comb}\ z\ w) \Rightarrow \mathsf{h}^\sharp\ (z - w)\ [w < 0, z]$ where $\mathsf{comb} : \mathsf{int} \to \mathsf{int} \to \mathsf{intpair}$ is a constructor. Note that $\mathsf{comb}\ z\ w$ is a value. Chaining processors can remove the occurrences of $\mathsf{g}^\sharp$, and end with (1) $\mathsf{f}^\sharp\ x\ y \Rightarrow \mathsf{h}^\sharp\ (x + y)\ [x \geq 0 \wedge y \geq 0]$, (2) $\mathsf{f}^\sharp\ x\ y \Rightarrow \mathsf{h}^\sharp\ (x - y)\ [x \geq 0 \wedge y < 0]$, (3) $\mathsf{f}^\sharp\ x\ y \Rightarrow \mathsf{h}^\sharp\ (-x + y)\ [x < 0 \wedge y \geq 0]$, and (4) $\mathsf{f}^\sharp\ x\ y \Rightarrow \mathsf{h}^\sharp\ (-x - y)\ [x < 0 \wedge y < 0]$.

5.3 Usable Rules

A key processor in the DP framework for full rewriting, which also applies in the innermost setting, is the reduction pair processor [21, Definition 25]. This processor is so powerful because it can potentially be used with a wide variety of different reduction pairs and does not require $\succ$ to be monotonic: we must show $s^\sharp \succ_\varphi t^\sharp$ or $s^\sharp \succeq_\varphi t^\sharp$ for all SDPs $s^\sharp \Rightarrow t^\sharp\ [\varphi]$, and $s \succeq_\varphi t$ for all rules $s \to t\ [\varphi]$, and we may then remove the SDPs oriented with $\succ$.

The challenge is that, no matter how small $\mathcal{P}$, we must orient all rules $\mathcal{R}$ in the DP problem – and all processors considered so far only modify the set $\mathcal{P}$ of SDPs. Fortunately, in the innermost setting, we can see that only some of the rules could potentially be relevant.

To illustrate the idea, consider a system $\mathcal{R}_{\mathsf{drop}}$ which includes at least the following rewrite rules and no other defining drop, dfoldr, cons, or nil:

$$\mathsf{drop}\ n\ l \to l\ [n \leq 0] \quad \mathsf{drop}\ n\ \mathsf{nil} \to \mathsf{nil}\ [\mathsf{t}, n] \quad \mathsf{drop}\ n\ (\mathsf{cons}\ x\ l) \to \mathsf{drop}\ (n-1)\ l\ [n > 0]$$

$$\mathsf{dfoldr}\ f\ y\ n\ \mathsf{nil} \to y \qquad [\mathsf{t}, n]$$
$$\mathsf{dfoldr}\ f\ y\ n\ (\mathsf{cons}\ x\ l) \to f\ x\ (\mathsf{dfoldr}\ f\ y\ n\ (\mathsf{drop}\ n\ l)) \qquad [\mathsf{t}, n]$$

where $\mathsf{drop} : \mathsf{int} \to \mathsf{alist} \to \mathsf{alist}$ and $\mathsf{dfoldr} : (\mathsf{a} \to \mathsf{b} \to \mathsf{b}) \to \mathsf{b} \to \mathsf{int} \to \mathsf{alist} \to \mathsf{b}$. To deal with the SDP $(\{\, \mathsf{dfoldr}^\sharp\ f\ y\ n\ (\mathsf{cons}\ x\ l) \Rightarrow \mathsf{dfoldr}^\sharp\ f\ y\ n\ (\mathsf{drop}\ n\ l)\ [\mathsf{t}, n] \,\}, \mathcal{R}_{\mathsf{drop}})$, we need a reduction pair processor; roughly, we should show that $\mathsf{cons}\ x\ l$ is somehow "greater" than $\mathsf{drop}\ n\ l$, which cannot be done by a subterm criterion or an integer mapping processor. However, since the variables are to be instantiated to normal forms, any rules other than the ones defining drop would not be used in a chain for this problem. Hence, only these three rules need to be oriented. This observation leads to the notion of usable rules [3, 13]. We base our formulation on a higher-order version [26] and start with two auxiliary definitions:

Definition 25.
For $\ell \to r\ [\varphi]$ where $\ell : A_1 \to \cdots \to A_n \to B$ for $B \in \mathcal{S}$, let $(\ell \to r\ [\varphi])^{\mathrm{ex}}$ be $\{\, \ell \to r\ [\varphi], \ell\ x_1 \to r\ x_1\ [\varphi], \ldots, \ell\ x_1 \cdots x_n \to r\ x_1 \cdots x_n\ [\varphi] \,\}$ where $x_1, \ldots, x_n$ are fresh variables. Let $\mathcal{R}^{\mathrm{ex}}$ denote $\bigcup_{\ell \to r\ [\varphi] \in \mathcal{R}} (\ell \to r\ [\varphi])^{\mathrm{ex}}$.

Definition 26.
The set of usable symbols in a term t with respect to a logical constraint φ, denoted by $\mathcal{U}_{\mathcal{F}}(t)[\varphi]$, is defined inductively as follows:
(1)
Suppose $t = \mathsf{f}\ t_1 \cdots t_n$ for $\mathsf{f} \in \mathcal{F}^\sharp$. If $t \in T(\mathcal{F}_\vartheta, \mathrm{Var}(\varphi))$, then $\mathcal{U}_{\mathcal{F}}(t)[\varphi] = \emptyset$; otherwise $\mathcal{U}_{\mathcal{F}}(t)[\varphi] = \{\, (\mathsf{f}, n) \,\} \cup \bigcup_{i=1}^{n} \mathcal{U}_{\mathcal{F}}(t_i)[\varphi]$.
(2)
Suppose $t = x\ t_1 \cdots t_n$ for $x \in \mathcal{V}$. If $n = 0$, then $\mathcal{U}_{\mathcal{F}}(t)[\varphi] = \emptyset$; otherwise $\mathcal{U}_{\mathcal{F}}(t)[\varphi] = \{\, \bot \,\}$.

Here $\bot$ is a special symbol indicating that potentially any symbol could be usable.

The set of usable symbols for a set $\mathcal{P}$ of SDPs and a set $\mathcal{R}$ of rewrite rules, denoted by $\mathcal{U}_{\mathcal{F}}(\mathcal{P}, \mathcal{R})$, is the smallest set U such that (1) $\mathcal{U}_{\mathcal{F}}(t^\sharp)[\varphi] \subseteq U$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P}$, and (2) $\mathcal{U}_{\mathcal{F}}(r)[\varphi] \subseteq U$ for all $(\mathsf{f}, n) \in U$ and $\mathsf{f}\ t_1 \cdots t_n \to r\ [\varphi] \in \mathcal{R}^{\mathrm{ex}}$. Then $\mathcal{U}_{\mathcal{F}}(\mathcal{P}, \mathcal{R})$ exists because it is the least pre-fixed point of an order-preserving mapping on a complete lattice.

We present usable rules as a class of DP processors:

Definition 27.
Given sets $\mathcal{P}$ of SDPs and $\mathcal{R}$ of rules: if $\bot \notin \mathcal{U}_{\mathcal{F}}(\mathcal{P}, \mathcal{R})$, the set $\mathcal{U}(\mathcal{P}, \mathcal{R})$ of usable rules is defined as $\{\, \mathsf{f}\ t_1 \cdots t_k \to r\ [\varphi] \in \mathcal{R} \mid (\mathsf{f}, n) \in \mathcal{U}_{\mathcal{F}}(\mathcal{P}, \mathcal{R}) \text{ and } k \leq n \,\}$; otherwise, $\mathcal{U}(\mathcal{P}, \mathcal{R})$ is undefined. A usable-rules processor assigns to a DP problem $(\mathcal{P}, \mathcal{R})$ with $\mathcal{NF}(\mathcal{Q}) \subseteq \mathcal{NF}(\mathcal{R})$ the singleton $\{\, (\mathcal{P}, \mathcal{U}(\mathcal{P}, \mathcal{R})) \,\}$.

Example 28.
We continue the discussion about drop and dfoldr. Consider the DP problem $(\mathcal{P}, \mathcal{R}_{\mathsf{drop}})$ where $\mathcal{P} = \{\, \mathsf{dfoldr}^\sharp\ f\ y\ n\ (\mathsf{cons}\ x\ l) \Rightarrow \mathsf{dfoldr}^\sharp\ f\ y\ n\ (\mathsf{drop}\ n\ l)\ [\mathsf{t}, n] \,\}$. We have $\mathcal{U}_{\mathcal{F}}(\mathsf{dfoldr}^\sharp\ f\ y\ n\ (\mathsf{drop}\ n\ l))[\mathsf{t}, n] = \{\, (\mathsf{dfoldr}^\sharp, 4), (\mathsf{drop}, 2) \,\}$ and can derive

$\mathcal{U}_{\mathcal{F}}(\mathcal{P},\mathcal{R}) = \{\, (\mathsf{dfoldr}^\sharp, 4), (\mathsf{drop}, 2), (\mathsf{nil}, 0) \,\}$. Hence, $\mathcal{U}(\mathcal{P},\mathcal{R})$ consists of the three rules defining drop.

5.4 Argument Filterings

Let us consider the role of $\bot$ in the definition of usable rules: $\mathcal{U}(\mathcal{P},\mathcal{R})$ is undefined if $\bot \in \mathcal{U}_{\mathcal{F}}(\mathcal{P},\mathcal{R})$ – which occurs if the right-hand side of some SDP in $\mathcal{P}$, or the right-hand side of any rule $\mathsf{f}\ t_1 \cdots t_n \to r\ [\varphi] \in \mathcal{R}^{\mathrm{ex}}$ with $(\mathsf{f}, n) \in \mathcal{U}_{\mathcal{F}}(\mathcal{P},\mathcal{R})$, is not a pattern. This is not a rare occasion in higher-order rewriting. For example, let us rework dfoldr into dfoldl:

$$\mathsf{dfoldl}\ f\ y\ n\ \mathsf{nil} \to y\ [\mathsf{t}, n] \qquad \mathsf{dfoldl}\ f\ y\ n\ (\mathsf{cons}\ x\ l) \to \mathsf{dfoldl}\ f\ (f\ y\ x)\ n\ (\mathsf{drop}\ n\ l)\ [\mathsf{t}, n]$$

where drop is defined by the same rules as before. Now we have $\mathsf{dfoldl}^\sharp\ f\ y\ n\ (\mathsf{cons}\ x\ l) \Rightarrow \mathsf{dfoldl}^\sharp\ f\ (f\ y\ x)\ n\ (\mathsf{drop}\ n\ l)\ [\mathsf{t}, n]$, which produces $\bot$.

However, observe that the problematic subterm $f\ y\ x$ and the decreasing subterm $\mathsf{drop}\ n\ l$ occur in different arguments of $\mathsf{dfoldl}^\sharp$. If we use a reduction pair that considers only the fourth argument, intuitively we can disregard any rules that may be used to reduce the second. This is formalized via an argument filtering [3, 26], which temporarily removes certain subterms from both sides of an ordering requirement before computing usable rules, often drastically reducing the number of usable rules that the reduction pair must consider.

Traditionally, argument filterings are defined for function symbols with a fixed number of arguments. Extending this notion to our curried setting imposes new technical challenges; e.g., if we filter away the second argument of $\mathsf{f} : A \to B \to C$, what type to use for the filtering of $\mathsf{f}\ x$, C or $B \to C$? Fortunately, this problem is solvable if we do not allow variables to occur at the head of an application in the result of a filtering. This is very different from the approach in [2], which heavily restricts what filtering can be used, to ensure that a variable of higher type and all its possible instances have the same filtering applied to them.

Definition 29.

We assume given a set $\mathcal{S}_1$ of sorts such that $\mathcal{S}_\vartheta \subseteq \mathcal{S}_1$, and a mapping μ from $\mathcal{T}$ to $\mathcal{S}_1$ such that $\mu(A) = A$ for all $A \in \mathcal{S}_\vartheta$. In addition, for each function symbol $\mathsf{f} : A_1 \to \cdots \to A_n \to B$ with $B \in \mathcal{S}$, we assume given a set $\mathrm{regard}(\mathsf{f}) \subseteq \{\, 1, \ldots, n \,\}$. Define:

$$\begin{aligned}\Sigma = \mathcal{F}_\vartheta \quad \cup \ \{ \quad & \mathsf{f}_m : \mu(A_{i_1}) \to \cdots \to \mu(A_{i_k}) \to \mu(A_{m+1} \to \cdots \to A_n \to B) \ \mid \\ & \mathsf{f} : A_1 \to \cdots \to A_n \to B \in \mathcal{F}^\sharp \text{ with } B \in \mathcal{S} \text{ and } 0 \le m \le n \text{ and} \\ & \mathrm{regard}(\mathsf{f}) \cap \{\, 1, \ldots, m \,\} = \{\, i_1, \ldots, i_k \,\} \text{ and } i_1 < \cdots < i_k \ \} \\ \cup \ \{ \quad & \bullet_{\mu(A)} : \mu(A) \mid A \in \mathcal{T} \ \}\end{aligned}$$

The argument filtering with respect to a logical constraint φ is a mapping π_φ from $T(\mathcal{F}^\sharp, \mathcal{V})$ to $T(\Sigma^\sharp, \{x : \mu(A) \mid x : A \in \mathcal{V}\})$, defined as follows:

(1)

$\pi_\varphi(\mathsf{f}\ t_1 \cdots t_n) = \mathsf{f}\ t_1 \cdots t_n$ if $\mathsf{f}\ t_1 \cdots t_n \in T(\mathcal{F}_\vartheta, \mathrm{Var}(\varphi))$ and has a base type

(2)

$\pi_\varphi(\mathsf{f}\ t_1 \cdots t_n) = \mathsf{f}_n\ \pi_\varphi(t_{i_1}) \cdots \pi_\varphi(t_{i_k})$ otherwise, where $\{\, i_1, \ldots, i_k \,\} = \mathrm{regard}(\mathsf{f}) \cap \{\, 1, \ldots, n \,\}$ and $i_1 < \cdots < i_k$

(3)

$\pi_\varphi(x) = x$ for $x \in \mathcal{V}$; and $\pi_\varphi(x\ t_1 \cdots t_n) = \bullet_{\mu(A)}$ if $x \in \mathcal{V}$ and $n > 0$

By definition, $\pi_\varphi(t) : \mu(A)$ for all $t : A$. Moreover, no subterm of $\pi_\varphi(t)$ has a variable at the head of an application, and all function symbols have a first-order type and occur maximally applied. Hence, $\pi_\varphi(t)$ is a (many-sorted) first-order term,

just written in applicative notation. For $\mathcal{S}_1$, we may for instance choose $\mathcal{S}_\vartheta \cup \{\,\mathsf{o}\,\}$, i.e., there is a single sort besides theory sorts.

Now we define usable rules with respect to an argument filtering:

Definition 30.

Given the mapping $\mathrm{regard}(\cdot), \mathcal{U}_\mathcal{F}(\mathsf{f}\ t_1 \cdots t_n)[\varphi]$ is redefined as $\{\,(\mathsf{f},n)\,\} \cup \bigcup_{i \in \mathrm{regard}(\mathsf{f}) \cap \{\,1,\ldots,n\,\}} \mathcal{U}_\mathcal{F}(t_i)[\varphi]$ in the case where $\mathsf{f}\ t_1 \cdots t_n \notin T(\mathcal{F}_\vartheta, \mathrm{Var}(\varphi))$. The definition in other cases is unchanged, and so is the definition of $\mathcal{U}_\mathcal{F}(\mathcal{P},\mathcal{R})$.

Let U_1 be $\{\,\pi_\varphi(\mathsf{f}\ t_1 \cdots t_n) \to \pi_\varphi(r)\ [\varphi] \,|\, (\mathsf{f},n) \in \mathcal{U}_\mathcal{F}(\mathcal{P},\mathcal{R})$ and $\mathsf{f}\ t_1 \cdots t_n \to r\ [\varphi] \in \mathcal{R}^{\mathrm{ex}}\,\}$, and U_2 be $\{\,\mathsf{f}_n\ x_{i_1} \cdots x_{i_k} \to y\ [y = \mathsf{f}\ x_1 \cdots x_n] \,|\, (\mathsf{f},n) \in \mathcal{U}_\mathcal{F}(\mathcal{P},\mathcal{R}), \mathsf{f} \in \mathcal{F}_\vartheta, n > 0, \mathsf{f} : A_1 \to \cdots \to A_n \to B$ with $B \in \mathcal{S}, \mathrm{regard}(\mathsf{f}) = \{\,i_1,\ldots,i_k\,\}$ and $i_1 < \cdots < i_k\,\}$.

We redefine $\mathcal{U}(\mathcal{P},\mathcal{R})$ as $U_1 \cup U_2$ if $\bot \notin \mathcal{U}_\mathcal{F}(\mathcal{P},\mathcal{R})$, and $p+i \in \mathrm{regard}(\mathsf{f})$ for all $(\mathsf{f},n) \in \mathcal{U}_\mathcal{F}(\mathcal{P},\mathcal{R})$, $\mathsf{f}\ s_1 \cdots s_p \to \mathsf{g}\ t_1 \cdots t_q\ [\varphi] \in \mathcal{R}$ where $p < n$ and $\mathsf{g} \in \mathcal{F}$, and $i \in \{\,1,\ldots,n-p\,\}$ such that $q+i \in \mathrm{regard}(\mathsf{g})$. Otherwise, $\mathcal{U}(\mathcal{P},\mathcal{R})$ is undefined.

That is, we consider usable rules only with respect to the positions that are not filtered away. The requirement that $p+i \in \mathrm{regard}(\mathsf{f})$ if (f,n) is a usable symbol and $p+i \le n$ is a technical limitation needed to ensure that applying a rewrite rule at the head of an application does not conflict with the argument filtering.

Next, we recall the notion of a constrained reduction pair from [21], but use a more liberal monotonicity requirement due to the first-order setting created by the filtering.

Definition 31.

A constrained relation R is a set of triples (s,t,φ) where s and t are (first-order) terms which have the same sort and φ is a logical constraint. We write $s\ R_\varphi\ t$ if $(s,t,\varphi) \in R$. A binary relation R' over terms is said to cover a constrained relation R if $s\ R_\varphi\ t$ implies that $(s\sigma)\!\downarrow_\kappa\ R'\ (t\sigma)\!\downarrow_\kappa$ for each substitution σ which respects φ.

A constrained reduction pair $(\succeq,\succ)$ is a pair of constrained relations where $\succeq$ is covered by some reflexive and transitive relation $\sqsupseteq$ such that $t_k \sqsupseteq t'_k$ implies $\mathsf{f}\ t_1 \cdots t_{k-1}\ t_k\ t_{k+1} \cdots t_n \sqsupseteq \mathsf{f}\ t_1 \cdots t_{k-1}\ t'_k\ t_{k+1} \cdots t_n$ (i.e., monotonicity) for all $\mathsf{f} \in \Sigma$ and $k \in \{\,1,\ldots,n\,\}$, $\succ$ is covered by some well-founded relation $\sqsupset$, and $\sqsupset\,;\sqsupseteq \subseteq \sqsupset^+$.

Having this, we can define reduction pair processors with respect to an argument filtering:

Definition 32.

A reduction pair processor with argument filterings assigns to a DP problem $(\mathcal{P},\mathcal{R})$ with $\mathcal{NF}(\mathcal{Q}) \subseteq \mathcal{NF}(\mathcal{R})$ the singleton $\{\,(\mathcal{P} \setminus \mathcal{P}', \mathcal{R})\,\}$ for some non-empty $\mathcal{P}' \subseteq \mathcal{P}$ if there exists a mapping $\mathrm{regard}(\cdot)$ and a constrained reduction pair $(\succeq,\succ)$ such that (1) $\pi_\varphi(s^\sharp) \succ_\varphi \pi_\varphi(t^\sharp)$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P}'$, (2) $\pi_\varphi(s^\sharp) \succeq_\varphi \pi_\varphi(t^\sharp)$ for all $s^\sharp \Rightarrow t^\sharp\ [\varphi] \in \mathcal{P} \setminus \mathcal{P}'$, and (3) $\ell \succeq_\varphi r$ for all $\ell \to r\ [\varphi] \in \mathcal{U}(\mathcal{P},\mathcal{R})$.

Example 33.

Consider dfoldl, with the following DP problem: $(\{\,\mathsf{dfoldl}^\sharp\ f\ y\ n\ (\mathsf{cons}\ x\ l) \Rightarrow \mathsf{dfoldl}^\sharp\ f\ (f\ y\ x)\ n\ (\mathsf{drop}\ n\ l)\ [\mathfrak{t}, n]\,\}, \mathcal{R})$ Let $\mathrm{regard}(\mathsf{dfoldl}^\sharp)$ be $\{\,4\,\}$, $\mathrm{regard}(\mathsf{drop})$ be $\{\,2\,\}$, and $\mathrm{regard}(\mathsf{cons})$ be $\{\,2\,\}$. We are obliged to prove (1) $\mathsf{dfoldl}^\sharp_4\ (\mathsf{cons}_2\ l) \succ_{\mathfrak{t},n} \mathsf{dfoldl}^\sharp_4\ (\mathsf{drop}_2\ l)$, (2) $\mathsf{drop}_2\ l \succeq_{n \le 0} l$, (3) $\mathsf{drop}_2\ \mathsf{nil}_0 \succeq_{\mathfrak{t},n} \mathsf{nil}_0$, and (4) $\mathsf{drop}_2\ (\mathsf{cons}_2\ l) \succeq_{n>0} \mathsf{drop}_2\ l$. The recursive path ordering for first-order LCTRSs [29] can be used to fulfill these obligations.

6 Universal Computability with Usable Rules

In this section, we study universal computability [21] with respect to innermost rewriting. This concept concerns a modular programming scenario where the LCSTRS represents a single module in a larger program, and corresponds to the

termination of a function in all "reasonable" uses, including unknown uses. We recall the notion of a hierarchical combination [31, 32, 33, 9] rephrased in terms of LCSTRSs:

Definition 34 ([21]).
An LCSTRS $\mathcal{R}_1$ is called an extension of a base system $\mathcal{R}_0$ if the two systems' interpretations of theory symbols coincide over all the theory symbols in common, and function symbols in $\mathcal{R}_0$ are not defined by any rewrite rule in $\mathcal{R}_1$. Given a base system $\mathcal{R}_0$ and an extension $\mathcal{R}_1$ of $\mathcal{R}_0$, the system $\mathcal{R}_0 \cup \mathcal{R}_1$ is called a hierarchical combination.

In a hierarchical combination, function symbols in the base system can occur in the extension, but cannot be (re)defined; one may think of $\mathcal{R}_0$ as an imported module.

Universal computability is defined on the basis of hierarchical combinations:

Definition 35 ([21]).
Given an LCSTRS $\mathcal{R}_0$ with a sort ordering $\succsim$, a term t is called universally computable if for each extension $\mathcal{R}_1$ of $\mathcal{R}_0$ and each extension $\succsim'$ of $\succsim$ to sorts in $\mathcal{R}_0 \cup \mathcal{R}_1$ (i.e., $\succsim'$ coincides with $\succsim$ over sorts in $\mathcal{R}_0$), t is $\mathbb{C}$-computable in $\mathcal{R}_0 \cup \mathcal{R}_1$ with $\succsim'$. $\mathcal{R}_0$ is called universally computable if all its terms are.

In summary, we consider passing $\mathbb{C}$-computable arguments to a defined symbol in $\mathcal{R}_0$ the "reasonable" way of calling the function. We use SDPs to establish universal computability:

This is a more general version of Theorem 14 and will be proved alongside it in Appendix A.2. Note that the original set of rules, $\mathcal{Q}$, in the definition of innermost chain is now $\mathcal{R}_0 \cup \mathcal{R}_1$. We modify the DP framework to prove universal computability for a fixed base system $\mathcal{R}_0$.

Definition 37.
A (universal) DP problem $(\mathcal{P}, \mathcal{R}, \mathtt{p})$ consists of a set $\mathcal{P}$ of SDPs, a set $\mathcal{R}$ of rewrite rules and a flag $\mathtt{p} \in \{\, \mathfrak{def}, \mathfrak{ind} \,\}$ (for definite or indefinite). A DP problem $(\mathcal{P}, \mathcal{R}, \mathtt{p})$ is finite if either (1) $\mathtt{p} = \mathfrak{def}$ and there exists no infinite innermost $(\mathcal{P}, \mathcal{R})$-chain (with $\mathcal{Q} \coloneqq \mathcal{R}_0$), or (2) $\mathtt{p} = \mathfrak{ind}$ and there exists no infinite innermost $(\mathcal{P}, \mathcal{R} \cup \mathcal{R}_1)$-chain for any extension $\mathcal{R}_1$ of $\mathcal{R}_0$ (with $\mathcal{Q} \coloneqq \mathcal{R}_0 \cup \mathcal{R}_1$).

DP processors are defined in the same way as before, now for universal DP problems. The goal is to show that $(\mathrm{SDP}(\mathcal{R}_0), \mathcal{R}_0, \mathfrak{ind})$ is finite, and the procedure for termination in Section 5 still works if we change the initialization accordingly. Unlike [21], here we have the advantage of usable rules: if $\mathcal{U}(\mathcal{P}, \mathcal{R})$ is defined, then $\mathcal{U}(\mathcal{P}, \mathcal{R} \cup \mathcal{R}_1) = \mathcal{U}(\mathcal{P}, \mathcal{R})$ for any extension $\mathcal{R}_1$ (provided all symbols in $\mathcal{P}$ and $\mathcal{R}$ are from $\mathcal{R}_0$, which is typically the case as DP processors generally do not introduce new symbols). Hence a usable-rules processor assigns to a DP problem $(\mathcal{P}, \mathcal{R}, \mathtt{p})$ the singleton $\{\, (\mathcal{P}, \mathcal{U}(\mathcal{P}, \mathcal{R}), \mathfrak{def}) \,\}$.

Even if usable-rules processors are not applicable, we may still use a reduction pair processor with an argument filtering: we do not have to orient the rules in $\mathcal{R}_1$ since they cannot be usable. Referring to Definition 32, a reduction pair processor now assigns to a DP problem $(\mathcal{P}, \mathcal{R}, \mathtt{p})$ the singleton $\{\, (\mathcal{P} \setminus \mathcal{P}', \mathcal{R}, \mathtt{p}) \,\}$ without changing the input flag because it does not permanently discard any rule. The other DP processors discussed in Section 5 apply to universal DP problems similarly; they just keep the input flag unchanged in the output.

Theorem 14.
An AFP system $\mathcal{R}$ is innermost (and therefore call-by-value) terminating if there exists no infinite innermost $(\mathrm{SDP}(\mathcal{R}), \mathcal{R})$-chain.

Theorem 36.

An accessible function passing system $\mathcal{R}_0$ with sort ordering $\succsim$ is universally computable if there exists no infinite innermost $(\mathrm{SDP}(\mathcal{R}_0), \mathcal{R}_0 \cup \mathcal{R}_1)$-chain for any extension $\mathcal{R}_1$ of $\mathcal{R}_0$ and extension $\succsim'$ of $\succsim$ to sorts in $\mathcal{R}_0 \cup \mathcal{R}_1$.

A.2 The Proofs of Theorems 14 and 36

For the properties of $\mathbb{C}$-computability, see Appendix A in the extended version of [13]. The following proofs are largely based on Appendix A.3 in [20]; here the novelty lies on the innermostness of chains. Note that undefined function symbols are $\mathbb{C}$-computable (see [20]).

Lemma 47.

Assume given an AFP system $\mathcal{R}_0$ with sort ordering $\succsim$, an extension $\mathcal{R}_1$ of $\mathcal{R}_0$ and a sort ordering $\succsim'$ which extends $\succsim$ over sorts in $\mathcal{R}_0 \cup \mathcal{R}_1$. For each defined symbol $\mathsf{f} : A_1 \to \cdots \to A_m \to B$ in $\mathcal{R}_0$ where B is a sort, if $\mathsf{f}\ s_1 \cdots s_m$ is not $\mathbb{C}$-computable in $\mathcal{R}_0 \cup \mathcal{R}_1$ with $\succsim'$ but s_i is for all i, then there exist an SDP $\mathsf{f}^\sharp\ {s'}_1 \cdots {s'}_m \Rightarrow \mathsf{g}^\sharp\ t_1 \cdots t_n\ [\varphi] \in \mathrm{SDP}(\mathcal{R}_0)$, a substitution σ and a natural number p such that (1) $s_i \xrightarrow{\mathrm{i}}{}^*_{\mathcal{R}_0 \cup \mathcal{R}_1} s'_i\sigma$ and $s'_i\sigma$ is in normal form for all $i \le p$, (2) s'_i is a fresh variable (s'_i does not occur in s'_j for any $j \neq i$) and $\sigma(s'_i) = s_i$ for all $i > p$, (3) σ respects φ, and (4) $(\mathsf{g}\ t_1 \cdots t_n)\sigma = \mathsf{g}\ (t_1\sigma) \cdots (t_n\sigma)$ is not $\mathbb{C}$-computable in $\mathcal{R}_0 \cup \mathcal{R}_1$ with $\succsim'$ but $u\sigma$ is for each proper subterm u of $\mathsf{g}\ t_1 \cdots t_n$.

Proof.

If the only reducts of $\mathsf{f}\ s_1 \cdots s_m$ were values or $\mathsf{f}\ {s'}_1 \cdots {s'}_m$ with $s_i \xrightarrow{\mathrm{i}}{}^*_{\mathcal{R}_0 \cup \mathcal{R}_1} s'_i$ for all i then $\mathsf{f}\ s_1 \cdots s_m$ would be computable. Hence, there exist $\mathsf{f}\ {s'}_1 \cdots {s'}_p \to r\ [\varphi] \in \mathcal{R}_0$ (f cannot be defined in $\mathcal{R}_1$) and σ' such that $s_i \xrightarrow{\mathrm{i}}{}^*_{\mathcal{R}_0 \cup \mathcal{R}_1} s'_i\sigma'$ and $s'_i\sigma'$ is in normal form for all $i \le p$, and σ' respects φ; $(r\sigma')\ s_{p+1} \cdots s_m$ is thus a reduct of $\mathsf{f}\ s_1 \cdots s_m$. At least one such reduct is uncomputable. Let $(r\sigma')\ s_{p+1} \cdots s_m$ be uncomputable, and therefore so is $r\sigma'$. Also all $\sigma'(x)$ are computable, since these are accessible subterms of some s'_i, since $\mathcal{R}_0$ is AFP.

Take a minimal subterm $a\ t_1 \cdots t_q$ of r such that $t\sigma'$ is uncomputable. By minimality, each $t_i\sigma'$ is computable. Hence, a cannot be a variable, value or constructor, as then $\sigma'(x)$ would be computable, implying computability of $t\sigma'$. Hence, a is a defined symbol g.

We have $\mathsf{f}^\sharp\ {s'}_1 \cdots {s'}_p\ x_{p+1} \cdots x_m \Rightarrow \mathsf{g}^\sharp\ t_1 \cdots t_q\ y_{q+1} \cdots y_n\ [\varphi] \in \mathrm{SDP}(\mathcal{R}_0)$. Because $t\sigma'$ is uncomputable, there exist computable terms ${t'}_{q+1}, \ldots, {t'}_n$ such that $(t\sigma')\ {t'}_{q+1} \cdots {t'}_n = \mathsf{g}\ (t_1\sigma') \cdots (t_q\sigma')\ {t'}_{q+1} \cdots {t'}_n$ is uncomputable. Let σ be such that $\sigma(x_i) = s_i$ for all $i > p$, $\sigma(y_i) = t'_i$ for all $i > q$, and $\sigma(z) = \sigma'(z)$ for any other z. Let s'_i denote x_i for $i > p$ and t_i denote y_i for $i > q$; then $\mathsf{f}^\sharp\ {s'}_1 \cdots {s'}_m \Rightarrow \mathsf{g}^\sharp\ t_1 \cdots t_n\ [\varphi]$, σ and p satisfy all requirements. ◀

Corollary 48.

Given an AFP system $\mathcal{R}_0$ with sort ordering $\succsim$, an extension $\mathcal{R}_1$ of $\mathcal{R}_0$ and a sort ordering $\succsim'$ which extends $\succsim$ over sorts in $\mathcal{R}_0 \cup \mathcal{R}_1$, for each defined symbol $\mathsf{f} : A_1 \to \cdots \to A_m \to B$ in $\mathcal{R}_0$ (B a sort), if $\mathsf{f}\ s_1 \cdots s_m$ is not $\mathbb{C}$-computable in $\mathcal{R}_0 \cup \mathcal{R}_1$ with $\succsim'$ but each s_i is, there exists an infinite innermost $(\mathrm{SDP}(\mathcal{R}_0), \mathcal{R}_0 \cup \mathcal{R}_1)$-chain $(\mathsf{f}^\sharp\ {s'}_1 \cdots {s'}_m \Rightarrow t^\sharp\ [\varphi], \sigma), \ldots$

Proof.

Repeatedly applying Lemma 47 (on $\mathsf{f}\ s_1 \cdots s_m$, then on $\mathsf{g}\ (t_1\sigma) \cdots (t_n\sigma)$ and so on) we get three infinite sequences: one of SDPs in $\mathrm{SDP}(\mathcal{R}_0)$ $(\mathsf{f}_i^\sharp\ s_{i,1} \cdots s_{i,m_i} \Rightarrow \mathsf{f}_{i+1}^\sharp\ t_{i,1} \cdots t_{i,n_i}\ [\varphi_i])_i$ where $\mathsf{f}_0 = \mathsf{f}$, one of substitutions $(\sigma_i)_i$ and one of natural numbers $(p_i)_i$. Since $\xrightarrow{\mathrm{i}}_{\mathcal{R}_0 \cup \mathcal{R}_1}$ is exactly $\xrightarrow{\mathcal{R}_0 \cup \mathcal{R}_1}_{\mathcal{R}_0 \cup \mathcal{R}_1}$, combining the SDPs and the substitutions almost gives us an infinite innermost $(\mathrm{SDP}(\mathcal{R}_0), \mathcal{R}_0 \cup \mathcal{R}_1)$-chain, except that $s_{i,j}\sigma_i$ may not be in normal form if $j > p_i$.

We define $(\sigma'_i)_i$ so that, together with the SDPs, we obtain an infinite chain. For all i, j with $p_i < j \leq m_i$, $s_{i,j}$ is a fresh variable and $\sigma_i(s_{i,j})$ is $\mathbb{C}$-computable. Let $\sigma'_i(x)$ be $\sigma_i(x)$ for $x \in \mathcal{V} \setminus \{ s_{i,p_i+1}, \ldots, s_{i,m_i} \}$. To define $\sigma'_i(s_{i,j})$ for $j > p_i$, let k_0 be j and for $l \geq 0$, if k_l exists and $k_l > p_{i+l}$, let k_{l+1} be the index (if any) with $s_{i+l,k_l} = t_{i+l,k_{l+1}}$. There are two options:
(1)

If $k_l > p_{i+l}$ for all l such that k_l is defined, regardless of whether the sequence is finite, $\sigma_{i+l}(s_{i+l,k_l}) = \sigma_{i+l}(t_{i+l,k_{l+1}}) = \sigma_{i+l+1}(s_{i+l+1,k_{l+1}})$ for all l such that k_{l+1} is defined. Let u be an arbitrary normal form of $\sigma_i(s_{i,j})$, and let $\sigma'_{i+l}(s_{i+l,k_l})$ be u for all l where k_l is defined.
(2)

Otherwise, the sequence $(k_l)_l$ is finite, and $k_q \leq p_{i+q}$ where k_q is the last in the sequence. Then $s_{i+q,k_q}\sigma_{i+q}$ is in normal form. Let $\sigma'_{i+l}(s_{i+l,k_l})$ be $s_{i+q,k_q}\sigma_{i+q}$ for all $l < q$.

Hence, we have built an infinite innermost $(\mathrm{SDP}(\mathcal{R}_0), \mathcal{R}_0 \cup \mathcal{R}_1)$-chain. ◀

Theorem 14. [Restated, see original statement.]
An AFP system $\mathcal{R}$ is innermost (and therefore call-by-value) terminating if there exists no infinite innermost $(\mathrm{SDP}(\mathcal{R}), \mathcal{R})$-chain.

Proof.
Towards a contradiction, assume that $\mathcal{R}$ is not innermost terminating. Then there is an uncomputable (not $\mathbb{C}$-computable) term u. Take a minimal subterm s of u that is uncomputable; then $u = \mathsf{f}\ s_1 \cdots s_k$ where f is defined and each s_i is computable. Let $\mathsf{f} : A_1 \to \cdots \to A_m \to B$ for $B \in \mathcal{S}$. Since $s = \mathsf{f}\ s_1 \cdots s_k$ is uncomputable, there exist computable $s_{k+1}, \ldots, s_m$ with $s\ s_{k+1} \cdots s_m = \mathsf{f}\ s_1 \cdots s_m$ uncomputable. By Corollary 48 (with $\mathcal{R}_1 = \emptyset$), there is an infinite innermost $(\mathrm{SDP}(\mathcal{R}), \mathcal{R})$-chain, giving the required contradiction. ◀

Theorem 36. [Restated, see original statement.]
An accessible function passing system $\mathcal{R}_0$ with sort ordering $\succsim$ is universally computable if there exists no infinite innermost $(\mathrm{SDP}(\mathcal{R}_0), \mathcal{R}_0 \cup \mathcal{R}_1)$-chain for any extension $\mathcal{R}_1$ of $\mathcal{R}_0$ and extension $\succsim'$ of $\succsim$ to sorts in $\mathcal{R}_0 \cup \mathcal{R}_1$.

Proof.
Towards a contradiction, assume that $\mathcal{R}_0$ is not universally computable. There exist an extension $\mathcal{R}_1$ of $\mathcal{R}_0$, sort ordering $\succsim'$ which extends $\succsim$ over sorts in $\mathcal{R}_0 \cup \mathcal{R}_1$ and term u of $\mathcal{R}_0$ that is not $\mathbb{C}$-computable in $\mathcal{R}_0 \cup \mathcal{R}_1$ with $\succsim'$. We consider $\mathbb{C}$-computability in $\mathcal{R}_0 \cup \mathcal{R}_1$. Take a minimal uncomputable subterm s of u; then s must take the form $\mathsf{f}\ s_1 \cdots s_k$ with f a defined symbol in $\mathcal{R}_0$ and each s_i computable. Let $\mathsf{f} : A_1 \to \cdots \to A_m \to B$. Because s is uncomputable, there exist computable $s_{k+1}, \ldots, s_m$ of $\mathcal{R}_0 \cup \mathcal{R}_1$ with $s\ s_{k+1} \cdots s_m = \mathsf{f}\ s_1 \cdots s_m$ uncomputable. Corollary 48 gives the required chain to obtain a contradiction. ◀

References

References[1] B. Alarcón, R. Gutiérrez, and S. Lucas. Context-sensitive dependency pairs. IC, 208(8):922–968, 2010. doi:10.1016/J.IC.2010.03.003. [2] T. Aoto and T. Yamada. Argument filterings and usable rules for simply typed dependency pairs. In S. Ghilardi and R. Sebastiani, editors, Proc. FroCoS, pages 117–132, 2009. doi:10.1007/978-3-642-04222-5_7. [3] T. Arts and J. Giesl. Termination of term rewriting using dependency pairs. TCS, 236(1–2):133–178, 2000. doi:10.1016/S0304-3975(99)00207-8. [4] D. Beyer, A. Cimatti, A. Griggio, M. E. Keremoglu, and R. Sebastiani. Software model checking via large-block encoding. In A. Biere and C. Pixley, editors, Proc. FMCAD, pages 25–32, 2009. doi:10.1109/FMCAD.2009.5351147. [5] M. Codish, C. Fuhs, J. Giesl, and P. Schneider-Kamp. Lazy abstraction for size-change termination. In C. G. Fermüller and A. Voronkov,

editors, Proc. LPAR (Yogyakarta), pages 217–232, 2010. doi:10.1007/978-3-642-16242-8_16. [6] Community. Termination competition (TermCOMP). URL: https://termination-portal.org/wiki/Termination_Competition. [7] Community. The termination problem database (TPDB). URL: https://github.com/TermCOMP/TPDB. [8] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In C.R. Ramakrishnan and J. Rehof, editors, Proc. TACAS, pages 337–340, 2008. doi:10.1007/978-3-540-78800-3_24. [9] N. Dershowitz. Hierarchical termination. In N. Dershowitz and N. Lindenstrauss, editors, Proc. CTRS, pages 89–105, 1995. doi:10.1007/3-540-60381-6_6. [10] S. Falke and D. Kapur. A term rewriting approach to the automated termination analysis of imperative programs. In R. A. Schmidt, editor, Proc. CADE, pages 277–293, 2009. doi:10.1007/978-3-642-02959-2_22. [11] S. Falke, D. Kapur, and C. Sinz. Termination analysis of C programs using compiler intermediate languages. In M. Schmidt-Schauß, editor, Proc. RTA, pages 41–50, 2011. doi:10.4230/LIPIcs.RTA.2011.41. [12] C. Fuhs, J. Giesl, M. Plücker, P. Schneider-Kamp, and S. Falke. Proving termination of integer term rewriting. In R. Treinen, editor, Proc. RTA, pages 32–47, 2009. doi:10.1007/978-3-642-02348-4_3. [13] C. Fuhs and C. Kop. A static higher-order dependency pair framework. In L. Caires, editor, Proc. ESOP, pages 752–782, 2019. doi:10.1007/978-3-030-17184-1_27. [14] C. Fuhs, C. Kop, and N. Nishida. Verifying procedural programs via constrained rewriting induction. ACM TOCL, 18(2):14:1–14:50, 2017. doi:10.1145/3060143. [15] J. Giesl, C. Aschermann, M. Brockschmidt, F. Emmes, F. Frohn, C. Fuhs, J. Hensel, C. Otto, M. Plücker, P. Schneider-Kamp, T. Ströder, S. Swiderski, and R. Thiemann. Analyzing program termination and complexity automatically with AProVE. JAR, 58(1):3–31, 2017. doi:10.1007/s10817-016-9388-y. [16] J. Giesl, M. Raffelsieper, P. Schneider-Kamp, S. Swiderski, and R. Thiemann. Automated termination proofs for Haskell by term rewriting. ACM TOPLAS, 33(2):7:1–7:39, 2011. doi:10.1145/1890028.1890030. [17] J. Giesl, T. Ströder, P. Schneider-Kamp, F. Emmes, and C. Fuhs. Symbolic evaluation graphs and term rewriting: a general methodology for analyzing logic programs. In A. King, editor, Proc. PPDP, pages 1–12, 2012. doi:10.1145/2370776.2370778. [18] J. Giesl, R. Thiemann, and P. Schneider-Kamp. The dependency pair framework: combining techniques for automated termination proofs. In F. Baader and A. Voronkov, editors, Proc. LPAR, pages 301–331, 2005. doi:10.1007/978-3-540-32275-7_21. [19] J. Giesl, R. Thiemann, P. Schneider-Kamp, and S. Falke. Mechanizing and improving dependency pairs. JAR, 37(3):155–203, 2006. doi:10.1007/s10817-006-9057-7. [20] L. Guo, K. Hagens, C. Kop, and D. Vale. Higher-order constrained dependency pairs for (universal) computability. pre-publication Arxiv copy of [21] with additional appendix. doi:10.48550/arXiv.2406.19379. [21] L. Guo, K. Hagens, C. Kop, and D. Vale. Higher-order constrained dependency pairs for (universal) computability. In R. Královič and A. Kučera, editors, Proc. MFCS, pages 57:1–57:15, 2024. doi:10.4230/LIPIcs.MFCS.2024.57. [22] L. Guo and C. Kop. Higher-order LCTRSs and their termination. In S. Weirich, editor, Proc. ESOP, pages 331–357, 2024. doi:10.1007/978-3-031-57267-8_13. [23] N. Hirokawa and A. Middeldorp. Automating the dependency pair method. IC, 199(1-2):172–199, 2005. doi:10.1016/J.IC.2004.10.004. [24] N. Hirokawa and A. Middeldorp. Tyrolean Termination Tool: techniques and features. IC, 205(4):474–511, 2007. doi:10.1016/j.ic.2006.08.010. [25] C. Kop. WANDA – a higher order termination tool (system description). In Z. M. Ariola, editor, Proc. FSCD, pages 36:1–36:19, 2020. doi:10.4230/LIPICS.FSCD.2020.36. [26] C. Kop. Cutting a proof into bite-sized chunks: incrementally proving termination in higher-order term rewriting. In A. P. Felty, editor, Proc. FSCD, pages 1:1–1:17, 2022. doi:10.4230/LIPIcs.FSCD.2022.1. [27] C. Kop et al. The Cora analyzer. URL: https://github.com/hezzel/cora. [28] C. Kop et al. hezzel/cora: FSCD 2025. doi:10.5281/zenodo.15318964. [29] C. Kop and N. Nishida. Term rewriting with logical constraints. In P. Fontaine, C. Ringeissen, and R. A. Schmidt, editors, Proc. FroCoS, pages 343–358, 2013. doi:10.1007/978-3-642-40885-4_24. [30] C. Kop and F. van Raamsdonk. Dynamic dependency pairs for algebraic functional systems. LMCS, 8(2), 2012. doi:10.2168/LMCS-8(2:10)2012. [31] M. R. K. Krishna Rao. Completeness of hierarchical combinations of term rewriting systems. In R. K. Shyamasundar, editor, Proc. FSTTCS, pages 125–138, 1993. doi:10.1007/3-540-57529-4_48. [32] M. R. K. Krishna Rao. Simple termination of hierarchical combinations of term rewriting systems. In M. Hagiya and J. C. Mitchell, editors, Proc. TACS, pages 203–223, 1994. doi:10.1007/3-540-57887-0_97. [33] M. R. K. Krishna Rao. Semi-completeness of hierarchical and super-hierarchical combinations of term rewriting systems. In P. D. Mosses, M. Nielsen, and M. I. Schwartzbach, editors, Proc. CAAP, pages 379–393, 1995. doi:10.1007/3-540-59293-8_208. [34] K. Kusakari, Y. Isogai, M. Sakai, and F. Blanqui. Static dependency pair method based on strong computability for higher-order rewrite systems. IEICE Trans. Inf. Syst., E92.D(10):2007–2015, 2009. doi:10.1587/transinf.E92.D.2007. [35] K. Kusakari and M. Sakai. Enhancing dependency pair method using strong computability in simply-typed term rewriting. AAECC, 18(5):407–431,

2007. doi:10.1007/s00200-007-0046-9. [36] K. Kusakari and M. Sakai. Static dependency pair method for simply-typed term rewriting and related techniques. IEICE Trans. Inf. Syst., E92.D(2):235–247, 2009. doi:10.1587/transinf.E92.D.235. [37] C. S. Lee, N. D. Jones, and A. M. Ben-Amram. The size-change principle for program termination. In C. Hankin and D. Schmidt, editors, Proc. POPL, pages 81–92, 2001. doi:10.1145/360204.360210. [38] S. Lucas. Context-sensitive computations in functional and functional logic programs. JFLP, 1998(1), 1998. URL: http://danae.uni-muenster.de/lehre/kuchen/JFLP/articles/1998/A98-01/A98-01.html. [39] C. Otto, M. Brockschmidt, C. von Essen, and J. Giesl. Automated termination analysis of Java bytecode by term rewriting. In C. Lynch, editor, Proc. RTA, pages 259–276, 2010. doi:10.4230/LIPIcs.RTA.2010.259. [40] S. Suzuki, K. Kusakari, and F. Blanqui. Argument filterings and usable rules in higher-order rewrite systems. IPSJ Online Trans., 4:114–125, 2011. doi:10.2197/ipsjtrans.4.114. [41] R. Thiemann. The DP framework for proving termination of term rewriting. PhD thesis, RWTH Aachen University, Germany, 2007. URL: http://darwin.bth.rwth-aachen.de/opus3/volltexte/2007/2066/. [42] R. Thiemann and J. Giesl. The size-change principle and dependency pairs for termination of term rewriting. AAECC, 16(4):229–270, 2005. doi:10.1007/S00200-005-0179-7. [43] J. van de Pol. Termination proofs for higher-order rewrite systems. In J. Heering, K. Meinke, B. Möller, and T. Nipkow, editors, Proc. HOA, pages 305–325, 1993. doi:10.1007/3-540-58233-9_14. [44] F. van Raamsdonk. Translating logic programs into conditional rewriting systems. In L. Naish, editor, Proc. ICLP, pages 168–182, 1997. doi:10.7551/mitpress/4299.003.0018. [45] H. Zantema. Termination of term rewriting by semantic labelling. FI, 24(1/2):89–105, 1995. doi:10.3233/FI-1995-24124.